

Alexandra – a look-ahead SAT solver for And-Inverter graphs

by Andrei Oleynikov

Table of Contents

The Problem.....	3
Literature Review.....	3
The Method.....	4
The Algorithm: Consensus Driven Reverse Propagation (CDRP).....	4
Implementation: Alexandra.....	10
Evaluation.....	11
Dynamic cast.....	11
Deadlocks in the Thread Pool.....	11
Latch Propagation.....	12
Empty Handlers.....	12
Scheduling of Fold and Add tasks.....	12
Repeated branch visits.....	12
Memory leaks.....	12
Pulse structure and comparison.....	13
Pulse addition logic.....	13
Conclusion.....	13
Appendix.....	13
main.cpp.....	13
Solver.cpp.....	16
graph.h.....	27
UnitPulse.cpp.....	35
Stack.cpp.....	39
solver.h.....	40
Pulse.cpp.....	42
NTrue.cpp.....	58
NFalse.cpp.....	58
IPulse.cpp.....	59
AggregatedPulse.cpp.....	60
Utilities.h.....	71
ThreadPool.cpp.....	74
NAnd.cpp.....	77
NNot.cpp.....	78
NLatch.cpp.....	79
NVariable.cpp.....	80
Node.cpp.....	81
Propagate.cpp.....	81
StartPoint.cpp.....	82
NodeHandler.cpp.....	83
Accumulator.cpp.....	84
References and Acknowledgements.....	92

The Problem

Boolean Satisfiability Problem (or SAT for short) is a mathematical problem of identifying whether or not a given boolean function can return True. Being an NP-complete, it does not have a solution yet, however there are heuristic approaches to solving it in specific cases [1].

The practical applications of SAT are numerous: Boolean satisfiability can be used in process calculus to model potential deadlocks in concurrent systems [1]; SAT sweeping is a technique of electronic circuit optimisation, when SAT solver is used to identify logically equivalent branches on a circuit which are then merged into one [8][9][10]; By extending a solver with exclusive or function, it can be used to determine how difficult it is to decode a given encryption [5].

The most popular approach is called Conflict-Driven Clause Learning (CDCL)[3], is designed to work on functions in Conjunctive-Normal Form (CNF). This means that a problem can be represented as a set of independent OR clauses with up to three operands, all of which need to return True [2].

However, there are other forms of representation, for example And-Inverter Graph (AIG) [4]. This notation describes a problem as a set of logical gates (AND and NOT) and input/output variables. This form is more appropriate for circuit verification and optimisation [8][10], but there are not many solvers that work on it directly because there is a number of existing CNF-based have been developed for a really long time and so there are plenty very good implementations. Therefore, problems in AIG format are either translated into CNF [9] or solved using an adapted version of CDCL [8][10].

This paper will describe a new algorithm for SAT solving on And-Inverter graphs, based on AIGER format [4] and discuss the details of its implementation, in particular its design issues and logic errors.

Literature Review

Conflict-Driven Clause Learning is an algorithm that builds up on Davis-Putnam-Logemann-Loveland algorithm (DPLL). The way how they both work is through variable propagation and backtracking. [2][3]

In CNF problem is defined as a set of independent clauses. Each clause can have up to three variables (or negations of the variables). A clause is an OR function, so it is satisfied by at least one variable being true. Since variables are shared between different clauses, if a variable in one clause set to a value, it is supposed to be set to the same value in all clauses (which is known as Variable Propagation). [4][11]

In order to show that a problem is satisfiable or not, both CDCL and DPLL are trying to find a minimum satisfying configuration for all given clauses, while also propagating variables. As mentioned above clauses need at least one variable/negation set to true. If variables in a clause are unknown or false, the last variable must be set to true. Therefore, if variable is not the last or if there already variable(s) set to true, the variable in question is set to false. [2][3]

In both cases, solver keeps track of what clauses they visited and what values were assigned to variables. This is done so that solver can see contradictions (if a solver tries to set a variable to both

true and false, commonly referred to as conflict) in their justification and revert to previous clause to try different value. [2][3]

The difference between algorithms is that DPLL will backtrack to the decision one clause back to the current clause, and will keep going up until it either resolves the conflict or there will be no more alternative decisions (if problem is not satisfiable). [2] CDCL on the other hand will note the configuration of the variables that led to the conflict, turn it into a new clause, add it to the problem, and backtrack to the earliest decision before any of the variables in the new clause were visited. This allows to minimise backtracking (as the solver knows how far it needs to jump) and ensures that the solver will not create the same conflicts. [3]

This method works well on CNF problems and have been used in many state-of-the-art solvers, such as MiniSAT and Glucose. Because of this, software for circuit verification and logic synthesis often uses external SAT engines for verification and sweeping, which means that a model of the circuit (which is already an AIG) needs to be translated into CNF. [7][8][9][10]

There are attempts to create an AIG native solvers, but they still rely on CDCL as the most effective known algorithm. The problem is that in logic networks the clauses and variables are somewhat equivalent, meaning that a clause can feed its output (or its negation) into another clause. This means that values of clauses also needs to be propagated, however it is possible only when a clause receives all inputs (which is referred to as justification). This leads to creation of justification frontier (j-frontier for short) - a set of all nodes on a graph that have not received all their inputs. This means that the solver needs to backtrack in order to justify nodes in the frontier (which can be a bottle neck for a performance), but it also creates a short cut for proving that a circuit is not satisfiable, as if j-frontier can not be emptied any further then the problem can not return true. [8][10]

When a solver is used for SAT sweeping it is used along with a circuit simulation. This is when nodes in a given part of the graph receive a sequence of random inputs and return a sequence of corresponding outputs. This sequences are fed into the nodes connected to the first ones. If the output sequences of the node are then compared. The nodes with equal output sequences are then categorised as equivalent. The SAT solver is then used to prove that these are equivalent and can be merged. The calls to SAT engine are the most time consuming, and so it is important to either have an efficient engine and/or a simulation method that can find potentially equivalent branches, while ignoring the ones that can be easily disproved. [8][9][10]

The Method

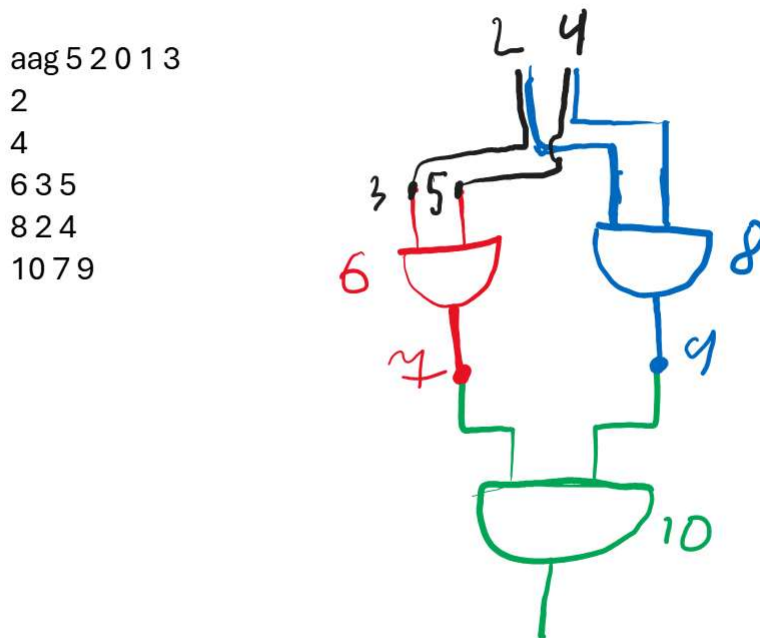
The Algorithm: Consensus Driven Reverse Propagation (CDRP)

The proposed algorithm is partially inspired by CDCL and circuit simulation. The idea is to re-imagine the concept of variable propagation on AIGs and enhance time performance by introducing parallelism into solving.

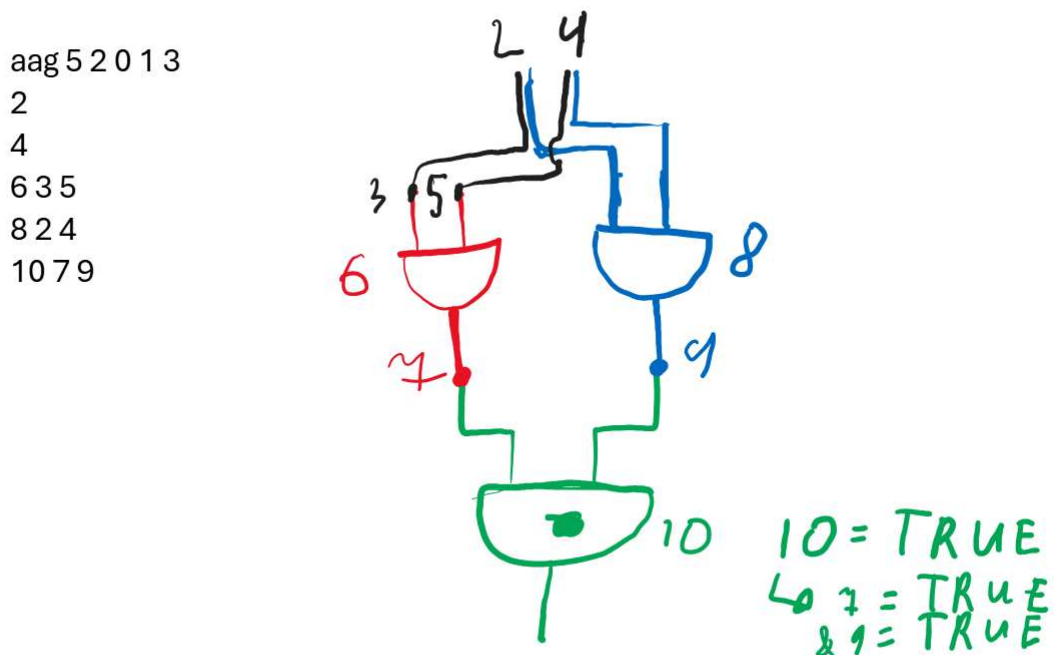
Let us consider logic gates AND and NAND. For whatever number of inputs (fanins) an output (fanout) of an AND gate can only be true if all fanins are true. Similarly, a NAND returns true whenever at least one fanin is false. Knowing the properties of the gates forming the problem and

that a satisfiable problem can return true, it is possible to work out the satisfying input values (and even an underlying pattern for all satisfying inputs) directly by propagating an output value.

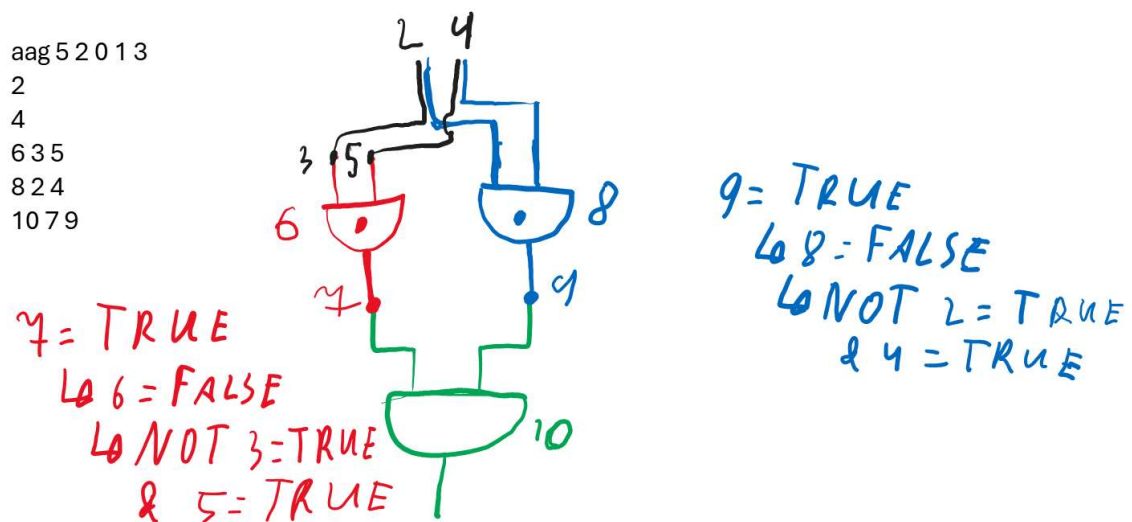
The steps for CDRP algorithm are as follows:



- Start at the fanout of the output node of the problem;

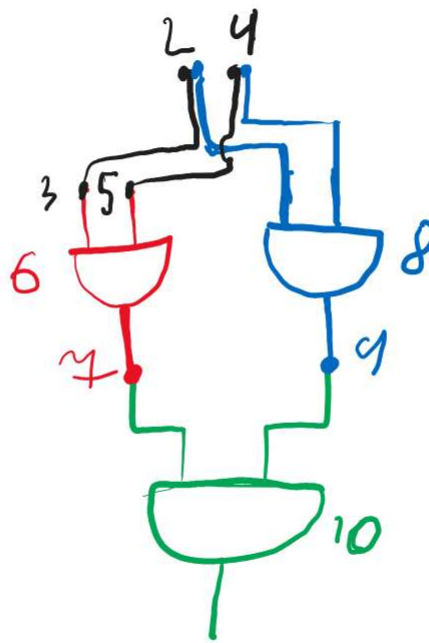


- Send a signal through the fanout of the node;
 - The signal should contain a list of NOT gates that it went through in reverse chronological order (the SAT trace);
 - A signal is assumed to be positive – this means that a signal is an output true for a given gate;
- Once the signal reaches a node it should propagate through its fanins into the next node;
 - If the node is an AND gate, then signal splits in two, which simultaneously travel to their respective fanins, containing exactly same traces;
 - If the node is an NOT gate, then the index of current node needs to be added to the signal's trace;



- Once signal reaches the variable node, its value must be added to a Semantic Tree;

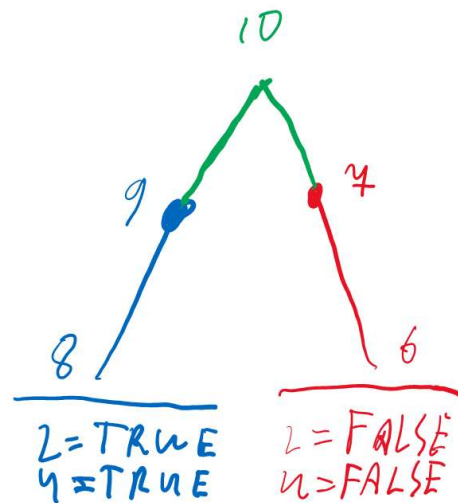
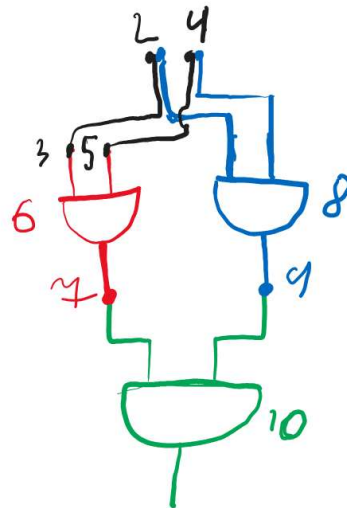
aag 5 2 0 1 3
 2
 4
 6 3 5
 8 2 4
 10 7 9



$$\begin{array}{c} \overline{(2 = 0)} \\ (4 = 0) \\ \hline (2 = 1) \\ (4 = 1) \end{array}$$

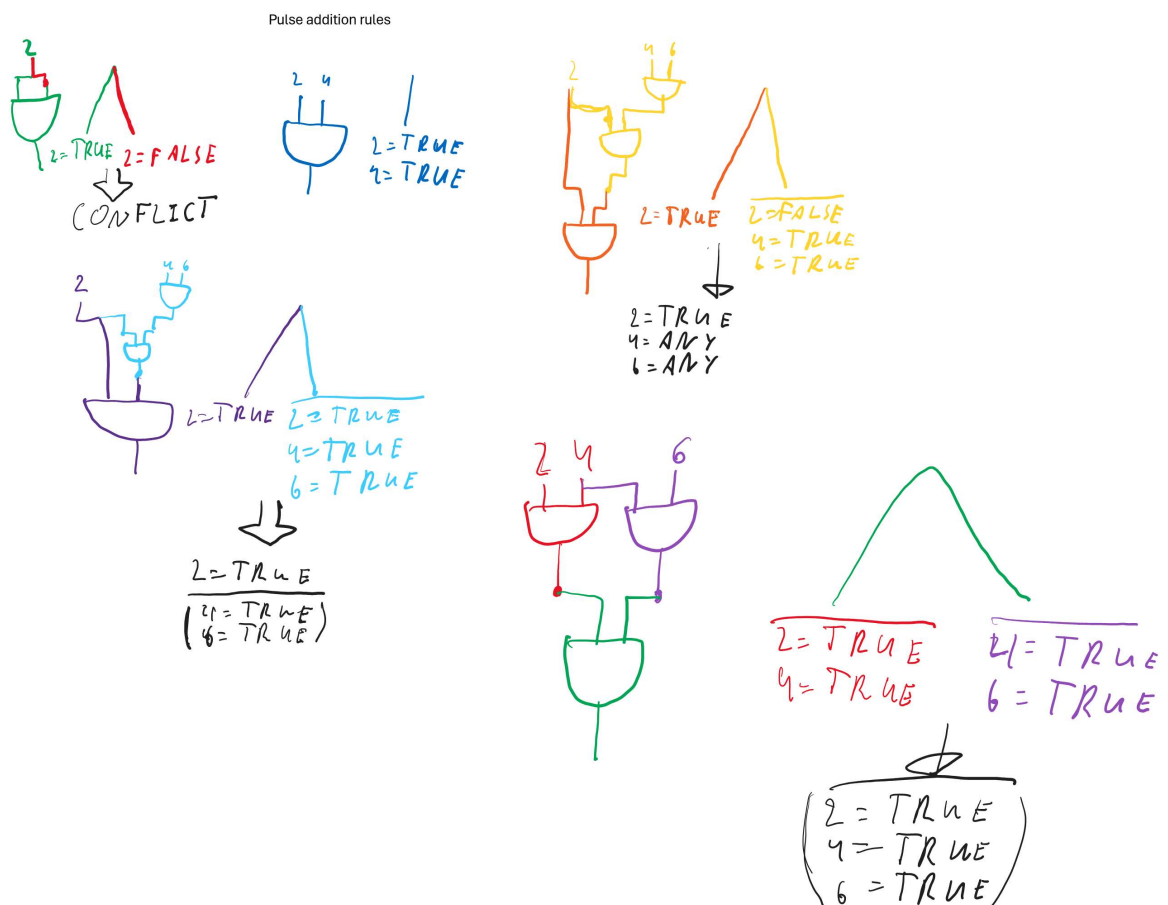
- If signal's trace is empty, then variable is set to true and added into a root node;
- If signal's trace contains NOT gate who's fanin is the variable, then variable is set to false and the NOT is removed from trace;
- While trace is not empty, create a child node (if does not exists already), add variable value to it or previous child (unless already added), remove last NOT gate from trace, if the trace is empty add current child to root node (unless already added);
- If a node contains a variable index/value pair and a new pair with the same index and opposite value is added, node contains a Conflict Pulse;

aag 5 2 0 1 3
 2
 4
 6 3 5
 8 2 4
 10 7 9



- Once all signals are added, merge nodes of the Semantic Tree together such that:
 - Index/value pairs in the node must form a Pulse – a pattern that describes all the satisfying solutions to a given branch/graph;
 - Master Pulse is considered positive, meaning it contains a satisfying input, and is formed from variables in the root node;
 - Child Pulses are considered negative, meaning that a satisfying input is any other then the one in the pulse, and are formed from child nodes;
 - Pulses can contain other pulses and lose index/value pairs;
 - Every pulse must contain unique variable/value pair (such that no variable index repeats);
 - Pulse from a child node must be added into a pulse from its parent.
 - If two pulses A and B stored together have common variable indexes, they are merged such that:
 - if common variables are set to the same values, added variables that are not present in unique variables from B into A;
 - otherwise, bundle pulses together into an Aggregated Pulse, as they can both be satisfied;
 - If a Child Pulse is added into Master Pulse, and a Master contains a variable that is present in a Child then:
 - if a value of a variable is the same in both, remove variable from Child;
 - otherwise, remove variable from Child and mark it as satisfied, meaning that variables inside of it can be of any values;
 - store Child in a Master;

- if a Child and a Master have all the same variables set to the same values, then conflict occurs;
- If a Child Pulse A is added into another Child Pulse B, contents of B are moved into A;
- If an Aggregate is added into any pulse it is either stored inside the pulse or merged if common variables present;
- When merged with Master Pulse, all pulses stored in Aggregate are merged with Master, potentially forming a new Aggregate inside;
- Aggregate cannot store another aggregate, they must be bundled into a pulse or their contents must form a new Aggregate (if common variables are present);



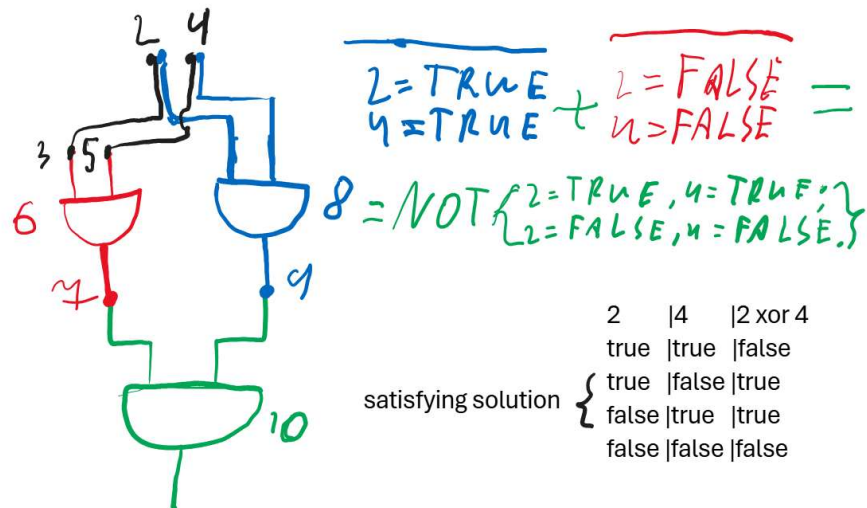
Once all nodes in Semantic Tree are merged, return final pulse;

- If a pulse is a Conflict Pulse, problem cannot be satisfied;
- else, problem is satisfiable.

```

aag 5 2 0 1 3
2
4
6 3 5
8 2 4
10 7 9

```



Implementation: Alexandra

The solver is written in C++ programming language for better performance. It is a command line application that takes the following inputs: number of threads (unsigned integer), path to an .aag file (a file in an ASCII AIGER format), and an assumption (a sequence of variable indexes in a format similar to AIGER where even number is variable set to true and odd is variable set to false).

The solution is organised into three namespaces:

- Alexandra – main namespace, contains Solver and Result classes;
- Utilities – contains thread pool, a single-linked tree stack implementation for trace and unique error classes;
- Graph – contains classes that form And-Inverter Graph model, Pulses, and an Accumulator class (which implements Semantic Tree).

When program is executed, it will parse the file, forming a syntax tree, which is equivalent to an AIG in our case. The parser is implemented as part of the Solver object's constructor. If the file does not have .aag ending or it doesn't follow AIGER format, parser will return invalid input error. The graph is stored as a map of node handlers (since clauses can be defined in any order it is necessary to create handlers which will then contain a pointer to a node), a pointer to an Accumulator object and an instance of a Start Point object, which contains pointers to the output node handlers (AIGER allows multiple nodes to be outputs). The parser can identify if a problem is satisfiable or not if its only output is set to 0 (always false) or 1 (always true), or not present at all. It will then through a respective error object from utility namespace.

The nodes store their own index and pointer(s) to their fanin clauses. There are four types of nodes: variable, AND, NOT, and a binary latch. A latch (flip-flop) node is part of AIGER format and it represents a storage mechanism (similar to real latches). It has a single fanout and a single fanin, which is a function that defines latch's next value (initial value is 0). Because of them, a propagated signal also has an integer value which says which node it came from previously. If the number is greater than an index of the next state function, then signal is propagated to the next function,

otherwise, a latch is added into an accumulator (as variable 0 which will either be ignored if in Child Pulse or cause a conflict if in Master). The variable nodes and latches store pointer to an Accumulator to add signal to the semantic tree.

Once a graph has been formed, the Start Point object schedules propagation of the output nodes to the thread pool. A vacant thread will pick up the task and start the process. Main thread will wait until propagation is complete. Every time a thread propagates the AND node it will schedule a new propagation task which will target a right fanin, while the thread will continue onto the left fanin.

When NOT node is encountered it is added to a SAT trace. The Trace is implemented as a single-linked tree stack, so that it can be easily copied and appended.

After propagation is done, Solver object will trigger a solve method in the Accumulator. Accumulator will schedule tasks for converting its nodes into pulses and then adding those pulses together.

There are three types of pulses:

- Unit Pulse – an index/value pair that can be true, false, any or conflicting;
- Pulse – a generic pulse that stores other pulse objects. Can be either positive or negative;
- Aggregate Pulse – an object that stores a bundle of negative pulses with common indexes and opposite values. This is necessary to preserve relationships when there are multiple values that cannot be true (for example XOR operation).

The pulses can be tested for equivalence (operator ==) or added (operator +). Two pulses are equivalent if they have unit pulses with common indexes. Addition operation either merges pulses or adds them into one another. The Result object in Alexandra namespace is wrapper for pulses that can be converted into a string, and is created once Accumulator finishes solving the semantic tree.

Evaluation

Unfortunately, the submitted solution had a vast amount of design, logical and syntactical errors which rendered it uncompileable. The rest of this section will focus on the logical and design problems which made this solver unusable, as well as how some of those problems were fixed. Appendix section contains comparison of the submitted code and the final solution (which compiles but does not return meaningful output).

Dynamic cast

Originally, the IPulse class which is a parent of all the pulses had methods that return pointers of type of its children, which is an illegal operation and cannot compile. This was solved by using dynamic cast function.

Deadlocks in the Thread Pool

In the first submission, locks were released instead of unlocking. Also, in some cases unlock clause could have been in a different scope or a method that acquired lock could have called another method that acquires the same lock. This was solved by restructuring methods in Thread Pool class and removing synchronisation from report task method.

Latch Propagation

When the propagation function was designed it was assumed that AIGER is hierarchical, meaning that nodes with higher index are defined after nodes with lower index. This however is not the case, and nodes can appear in any order. This is why, propagating into a latch next value function only when a previous node is greater then next function does not guarantee a correct behaviour. This was solved by replacing last node index parameter to another stack that keeps track of visited latches. If a latch occurs for a first time, then thread propagates into next function, otherwise assume latch to be false.

Empty Handlers

When reading the file, parser used to create empty handlers where there were supposed to be clauses. This was because when a parser encountered a negation of any node it would always create a new node handler for that node, instead of getting existing handler from the map and adding it into NOT node.

Scheduling of Fold and Add tasks

Accumulator used to schedule two types of tasks when solve method is called – Fold (turn Accumulator Node into a Pulse) and Add (add Pulse to another Pulse). This tasks were intertwined such that a Fold object would store its results in an Add object, and Add task would not be performed until all Folds were done (it would reschedule itself until all Folds were performed). Also, if a node had children it would schedule more folds and adds. This behaviour led to eternal scheduling of empty tasks that kept producing more tasks.

To solve this, Add class was removed and Fold was modified. In the new logic Accumulator Nodes have a children count and fold count and a pointer to their parent. When Fold is performed, child node adds its resultant pulse to a pulse of its parent and increment its fold count. When fold count equals to the children count then node schedules its own Fold. When solve method is called Accumulator schedules folding on the nodes with zero child count. When folding reaches the Master Pulse, solving is complete and value is returned.

Repeated branch visits

During propagation, thread can visit the same branch multiple times because it can be a fanin of multiple AND nodes, which means solver wastes more time then its actually needs. This is a fundamental design flaw that was not solved. The possible solution would be caching the repeatedly occurring nodes, so that results of their propagation could be reused in other clauses. Also, by marking already visited nodes, threads will not need to explore them again, which will reduce propagation time.

Memory leaks

During Pulse addition and propagation, solver produces lots of objects that are allocated at heap, rather than on a stack. This objects are passed by pointer and are not deleted, because in the original submission delete operation caused heap corruption. In the future, implementation must have an adequate memory management, so that it does not consume more memory then it needs.

Pulse structure and comparison

The Pulse objects contain a boolean variable that states whether they are positive or negative, and a vector of IPulse object pointers. This structure makes it difficult to compare pulses because there is no guarantee that a pulses will be in any order, as such pulse comparison can be of complexity $O(n^2)$. If pulses were sorted that could have reduced the complexity of comparison and even addition, as it would always be easy to find necessary unit pulse.

Another issue with comparison is that operator `==` is not enough, because there are multiple degrees of equivalence. Therefore, a pulse must have a new more flexible equivalence function and operators greater and less then for effective sorting.

Pulse addition logic

The pulse addition logic is wrong. It is over-bloated and unmaintainable, and there are number of mistakes.

First of all, Negative Pulses must store Aggregated Pulses (which they currently not) in order to accommodate for the cases when exclusive logic is fed into a not and clause.

Secondly, the Aggregate addition needs to be redesigned because as there is a race condition when it mistakes pulse types and tries to cast them into a different type.

Finally, the inter-pulse addition. The structure when a pulse object can store another pulse object (not be confused with aggregates) is wrong, because it leads to a dangerous many-to-many relationship which current solver does not accommodate for. This lead to the solver constantly having duplicate variables and incorrect addition results, which always lead to conflict result. This issue requires a complete redesign of the pulses, especially their addition.

Conclusion

Although a new algorithm does look promising, its current implementation is fundamentally unusable because of poor design decisions and unverifiable results. It needs better data structures and more optimised work allocation algorithms to work properly, but it does have potential to improve the SAT solving for circuit verification.

Appendix

This section contains comparison of the original submission and final version. The files that are not present were not modified.

main.cpp

Original

```
#include "Alexandra.h"
```

```
int main(int argc, char* argv[])
{
    std::string args = *argv;
    std::vector<std::string> arguments;
    auto start_p = args.begin();
    for (auto p = args.begin(); p != args.end(); p++)
    {
```

```

        if (*p == ' ')
        {
            arguments.push_back(std::string(start_p, p));
            start_p = p + 1;
        }
    }

    int number_of_threads = 0;
    std::string file_path;
    std::map<int, bool> assumption;

    if (arguments.size() == 1)
    {
        file_path = arguments.back();
    }
    else
    {
        try
        {
            number_of_threads = std::stoi(arguments[0]);
        }
        catch (std::invalid_argument* err)
        {
            std::cout << err << std::endl;
            return;
        }

        file_path = arguments[1];

        for (int i = 2; i < arguments.size(); i++)
        {
            int variable;
            try
            {
                variable = std::stoi(arguments[i]);
            }
            catch (std::invalid_argument* err)
            {
                std::cout << err << std::endl;
                return;
            }
            if (variable / 2 == 0)
            {
                assumption[variable] = true;
            }
            else
            {
                assumption[variable - 1] = false;
            }
        }
    }

    if (number_of_threads >= 0)
    {
        Utilities::ThreadPool::initialise(number_of_threads);
    }
    else
    {
        std::cout << "invalid input: number of threads cannot be
negative";
        return;
    }

    try

```

```

    {
        Alexandra::Solver solver(file_path);
        if (assumption.empty())
        {
            std::cout << (std::string)solver.solve() << std::endl;
        }
        else
        {
            std::cout << (std::string)solver.solve(assumption) <<
std::endl;
        }
    }
    catch (std::exception& err)
    {
        std::cout << err.what() << std::endl;
    }
}

```

Final:

```

#include "Alexandra.h"

int main(int argc, char* argv[])
{
    std::vector<std::string> arguments;
    for (int i = 1; i < argc; i++)
    {
        arguments.push_back(argv[i]);
    }

    if (arguments.empty())
    {
        std::cout << "please input number of threads (optional),
file path, and assumption (optional)" << std::endl;
        return 0;
    }

    int number_of_threads = 0;
    std::string file_path;
    std::map<int, bool> assumption;

    if (arguments.size() == 1)
    {
        file_path = arguments.back();
    }
    else
    {
        try
        {
            number_of_threads = std::stoi(arguments[0]);
        }
        catch (std::invalid_argument* err)
        {
            std::cout << err << std::endl;
            return 0;
        }

        file_path = arguments[1];

        for (int i = 2; i < arguments.size(); i++)
        {
            int variable;
            try

```

```

        {
            variable = std::stoi(arguments[0]);
        }
        catch (std::invalid_argument* err)
        {
            std::cout << err << std::endl;
            return 0;
        }
        if (variable / 2 == 0)
        {
            assumption[variable] = true;
        }
        else
        {
            assumption[variable - 1] = false;
        }
    }
}

if (number_of_threads >= 0)
{
    Utilities::ThreadPool::initialise(number_of_threads);
}
else
{
    std::cout << "invalid input: number of threads cannot be
negative";
    return 0;
}

try
{
    Alexandra::Solver solver(file_path);
    if (assumption.empty())
    {
        std::cout << (std::string)solver.solve() <<
std::endl;
    }
    else
    {
        std::cout <<
(std::string)solver.solve(assumption) << std::endl;
    }
}
catch (std::exception& err)
{
    std::cout << err.what() << std::endl;
}
}

```

Solver.cpp

Original:

```
#include "Alexandra.h"
```

```
namespace Alexandra
{
```

```

    Solver::Solver(std::string& file_path) :file(), nodes(), delimiter(' '),
    accumulator()
    {
        if (file_path.substr(file_path.size() - 4) == ".aag")

```



```

{
    file.open(file_path, std::ifstream::in);
    if (Solver::validate_header())
    {
        inputs();
        latches();
        outputs();
        ands();
        file.close();
    }
    else throw std::invalid_argument("invalid file format");
}
else
{
    throw std::invalid_argument("file is not accepted");
}
}

bool Solver::validate_header()
{
    std::string header;
    file >> header;
    std::stringstream stream(header);
    std::string token;
    std::vector<std::string> tokens;
    while(std::getline(stream, token, delimiter))
    {
        tokens.push_back(token);
    }

    if (tokens.size() != 6)
    {
        return false;
    }
    else
    {
        if (tokens.at(0) != "aag") return false;

        try
        {
            number_of_inputs = std::stoi(tokens.at(2));
            number_of_latches = std::stoi(tokens.at(3));
            number_of_outputs = std::stoi(tokens.at(4));
            number_of_ands = std::stoi(tokens.at(5));
        }
        catch (std::invalid_argument)
        {
            return false;
        }
    }
}

```

```

        return true;
    }

    void Solver::add_node(int node_index)
    {
        Graph::NodeHandler* vertice = new Graph::NodeHandler();
        if (node_index == 0)
        {
            Graph::NFalse* node_0 = new Graph::NFalse(&accumulator);
            vertice->set_node(node_0);
            nodes.emplace(node_index, vertice);
        }
        else if (node_index == 1)
        {
            Graph::NTrue* node_1 = new Graph::NTrue(&accumulator);
            vertice->set_node(node_1);
            nodes.emplace(node_index, vertice);
        }
        else
        {
            if (node_index % 2)
            {
                Graph::NNot* not_node = new
Graph::NNot(node_index, vertice);
                Graph::NodeHandler not_vertice = new
Graph::NodeHandler(not_node);

                nodes.emplace(node_index, not_vertice);
                nodes.emplace(node_index - 1, vertice);
            }
            else
            {
                nodes.emplace(node_index, vertice);
            }
        }
    }

    void Solver::inputs()
    {
        for (int i = 1; i <= number_of_inputs; i++)
        {
            std::string line;
            file>>line;
            int node_index;

            try
            {
                node_index = std::stoi(line);
            }
            catch (std::invalid_argument)
            {
                throw new std::invalid_argument("invalid file
format");
            }

            add_node(node_index);

            nodes.at(node_index).set_node(new
Graph::NVariable(node_index, &accumulator));
        }
    }

```

```

void Solver::latches()
{
    for (int i = 1; i <= number_of_latches; i++)
    {
        std::string line;
        file >> line;
        std::stringstream stream(line);

        int node_index;
        int next_state;

        try
        {
            std::string token_latch;
            std::getline(stream, token_latch, delimiter);
            node_index = std::stoi(token_latch);

            std::string token_next;
            std::getline(stream, token_next, delimiter);
            next_state = std::stoi(token_next);
        }
        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid file
format");
        }

        add_node(node_index);

        Graph::Node* node_next_state;

        if (!nodes.contains(next_state))
        {
            add_node(next_state);
        }

        node_next_state = &nodes.at(next_state);

        Graph::NLatch* latch = new Graph::NLatch(node_index,
next_state, node_next_state, &accumulator);
        add_node(node_index);

        nodes.at(next_state).set_node(latch);
    }
}

void Solver::outputs()
{
    if (number_of_outputs > 1)
    {
        Graph::Node** outputs = new Graph::Node*
[number_of_outputs];

        for (int i = 1; i <= number_of_outputs; i++)
        {
            std::string line;
            file >> line;
            int index;

            try
            {
                index = std::stoi(line);
            }

```

```

        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid
file format");
        }

        if (!nodes.contains(index))
        {
            add_node(index);
        }

        *outputs = &nodes.at(index);
        outputs++;
    }

    start_point = new Graph::StartPoint(outputs,
number_of_outputs);
    }
    else if (number_of_outputs == 1)
    {
        Graph::Node** outputs = new Graph::Node *
[number_of_outputs];

        std::string line;
        file >> line;
        int index;

        try
        {
            index = std::stoi(line);
        }
        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid file
format");
        }

        if (index == 1)
        {
            throw Utilities::AlwaysTrue();
        }
        else if (index == 0)
        {
            throw Utilities::AlwaysFalse();
        }
        else
        {
            if (!nodes.contains(index))
            {
                add_node(index);
            }

            *outputs = &nodes.at(index);
        }

        start_point = new Graph::StartPoint(outputs,
number_of_outputs);
    }
    else
    {
        throw Utilities::Unsolvable();
    }
}

```

```

void Solver::ands()
{
    for (int i = 1; i < number_of_ands; i++)
    {
        std::string line;
        file >> line;
        std::stringstream stream(line);

        std::string node_token;
        std::string left_token;
        std::string right_token;

        int node_index;
        int right_fanin;
        int left_fanin;

        try
        {
            std::getline(stream, node_token, delimiter);
            std::getline(stream, left_token, delimiter);
            std::getline(stream, right_token, delimiter);

            node_index = std::stoi(node_token);
            right_fanin = std::stoi(right_token);
            left_fanin = std::stoi(left_token);
        }
        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid file
format");
        }

        if (!nodes.contains(node_index))
        {
            add_node(node_index);
        }

        if (!nodes.contains(right_fanin))
        {
            add_node(right_fanin);
        }

        if (!nodes.contains(left_fanin))
        {
            add_node(left_fanin);
        }

        Graph::Node* node = new Graph::NAnd(node_index,
&nodes.at(left_fanin), &nodes.at(right_fanin));

        nodes.at(node_index).set_node(node);
    }
}

Result& Solver::solve()
{
    start_point->start();
    return *new Result(accumulator.solve());
}

Result& Solver::solve(std::map<int, bool> assumption)
{
    start_point->start();
    return *new Result(accumulator.solve(assumption));
}

```

```
}  
}
```

Final:

```
#include "Alexandra.h"
```

```
namespace Alexandra  
{
```

```
    Solver::Solver(std::string& file_path) :file(), nodes(), delimiter(' '),  
    accumulator()  
    {
```

```
        if (file_path.substr(file_path.size() - 4) == ".aag")  
        {
```

```
            file.open(file_path, std::ifstream::in);
```

```
            if (Solver::validate_header())  
            {
```

```
                inputs();
```

```
                latches();
```

```
                outputs();
```

```
                ands();
```

```
                file.close();
```

```
            }
```

```
            else throw std::invalid_argument("invalid file format -  
header");
```

```
        }
```

```
    else
```

```
    {
```

```
        throw std::invalid_argument("file is not accepted");
```

```
    }
```

```
}
```

```
bool Solver::validate_header()
```

```
{
```

```
    std::string header;
```

```
    std::getline(file, header);
```

```
    std::stringstream stream(header);
```

```
    std::string token;
```

```
    std::vector<std::string> tokens;
```

```
    while(std::getline(stream, token, delimiter))
```

```
    {
```

```
        tokens.push_back(token);
```

```
    }
```

```
    if (tokens.size() != 6)
```

```
    {
```

```
        return false;
```

```
    }
```

```
    else
```

```

{
    if (tokens.at(0) != "aag") return false;
    try
    {
        number_of_inputs = std::stoi(tokens.at(2));
        number_of_latches = std::stoi(tokens.at(3));
        number_of_outputs = std::stoi(tokens.at(4));
        number_of_and = std::stoi(tokens.at(5));
    }
    catch (std::invalid_argument)
    {
        return false;
    }
}

return true;
}

void Solver::add_node(int node_index)
{
    Graph::NodeHandler* vertice = new Graph::NodeHandler();
    if (node_index == 0)
    {
        Graph::NFalse* node_0 = new Graph::NFalse(&accumulator);
        vertice->set_node(node_0);
        nodes[node_index] = vertice;
    }
    else if (node_index == 1)
    {
        Graph::NTrue* node_1 = new Graph::NTrue(&accumulator);
        vertice->set_node(node_1);
        nodes[node_index] = vertice;
    }
    else
    {
        if (node_index % 2)
        {
            if (!nodes.contains(node_index-1))
            {
                add_node(node_index - 1);
            }

            Graph::NNot* not_node = new
Graph::NNot(node_index, nodes[node_index-1]);
            vertice->set_node(not_node);

            nodes[node_index] = vertice;

            std::cout << nodes[node_index]->node->me <<
std::endl;
        }
        else
        {
            nodes[node_index] = vertice;
        }
    }
}
}

```

```

void Solver::inputs()
{
    for (int i = 1; i <= number_of_inputs; i++)
    {
        std::string line;
        std::getline(file, line);
        std::cout << "line :" << line << std::endl;
        int node_index;

        try
        {
            node_index = std::stoi(line);
        }
        catch (std::invalid_argument)
        {
            throw new std::invalid_argument("invalid file
format");
        }

        add_node(node_index);

        nodes[node_index]->set_node(new
Graph::NVariable(node_index, &accumulator));

        std::cout << nodes[node_index]->node->me << std::endl;
    }
}

void Solver::latches()
{
    for (int i = 1; i <= number_of_latches; i++)
    {
        std::string line;
        std::getline(file, line);
        std::cout << "line :" << line << std::endl;
        std::stringstream stream(line);

        int node_index;
        int next_state;

        try
        {
            std::string token_latch;
            std::getline(stream, token_latch, delimiter);
            node_index = std::stoi(token_latch);

            std::string token_next;
            std::getline(stream, token_next, delimiter);
            next_state = std::stoi(token_next);
        }
        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid file
format");
        }

        add_node(node_index);

        Graph::Node* node_next_state;

        if (!nodes.contains(next_state))
        {
            add_node(next_state);
        }
    }
}

```



```

    }

    node_next_state = nodes[next_state];

    Graph::NLatch* latch = new Graph::NLatch(node_index,
next_state, node_next_state, &accumulator);

    nodes[node_index]->set_node(latch);

    std::cout << nodes[node_index]->node->me << std::endl;
    }
}

void Solver::outputs()
{
    if (number_of_outputs > 1)
    {
        Graph::Node** outputs = new Graph::Node*
[number_of_outputs];

        for (int i = 1; i <= number_of_outputs; i++)
        {
            std::string line;
            std::getline(file, line);
            std::cout << "line :" << line << std::endl;
            int index;

            try
            {
                index = std::stoi(line);
            }
            catch (std::invalid_argument)
            {
                throw std::invalid_argument("invalid
file format");
            }

            if (!nodes.contains(index))
            {
                add_node(index);
            }

            *outputs = nodes[index];
            outputs++;
        }

        start_point = new Graph::StartPoint(outputs,
number_of_outputs);
    }
    else if (number_of_outputs == 1)
    {
        Graph::Node** outputs = new Graph::Node *
[number_of_outputs];

        std::string line;
        std::getline(file, line);
        int index;

        try
        {
            index = std::stoi(line);
        }
        catch (std::invalid_argument)
        {

```

```

        throw std::invalid_argument("invalid file
format");
    }

    if (index == 1)
    {
        throw Utilities::AlwaysTrue();
    }
    else if (index == 0)
    {
        throw Utilities::AlwaysFalse();
    }
    else
    {
        if (!nodes.contains(index))
        {
            add_node(index);
        }

        *outputs = nodes[index];
    }

    start_point = new Graph::StartPoint(outputs,
number_of_outputs);
    }
    else
    {
        throw Utilities::Unsolvable();
    }
}

void Solver::ands()
{
    for (int i = 1; i <= number_of_and; i++)
    {
        std::string line;
        std::getline(file, line);
        std::cout << "line : " << line << std::endl;
        std::stringstream stream(line);

        std::string node_token;
        std::string left_token;
        std::string right_token;

        int node_index;
        int right_fanin;
        int left_fanin;

        try
        {
            std::getline(stream, node_token, delimiter);
            std::getline(stream, left_token, delimiter);
            std::getline(stream, right_token, delimiter);

            node_index = std::stoi(node_token);
            right_fanin = std::stoi(right_token);
            left_fanin = std::stoi(left_token);
        }
        catch (std::invalid_argument)
        {
            throw std::invalid_argument("invalid file
format");
        }
    }
}

```

```

        if (nodes[node_index] == NULL)
        {
            add_node(node_index);
        }

        if (nodes[right_fanin] == NULL)
        {
            add_node(right_fanin);
        }

        if (nodes[left_fanin] == NULL)
        {
            add_node(left_fanin);
        }

        Graph::NAnd* node = new Graph::NAnd(node_index,
nodes[left_fanin], nodes[right_fanin]);

        nodes[node_index]->set_node(node);

        std::cout << nodes[node_index]->node->me << std::endl;
    }
}

Result& Solver::solve()
{
    start_point->start();
    return *new Result(accumulator.solve());
}

Result& Solver::solve(std::map<int, bool> assumption)
{
    start_point->start();
    return *new Result(accumulator.solve(assumption));
}
}

```

graph.h

Original:

```

#pragma once
#include "utilities.h"

namespace Graph
{
    enum State
    {
        ANY, TRUE, FALSE, CONFLICT
    };

    struct IUnit
    {
        const State value;
        const int variable;
        IUnit(State, int);
    };

    struct IPulse
    {
        static enum PulseType {
            UnitPulse, Pulse, AggregatedPulse
        };
    };
}

```

```

const PulseType type;

IPulse(PulseType);

virtual Graph::AggregatedPulse* toAggregatedPulse();
virtual Graph::Pulse* toPulse();
virtual Graph::UnitPulse* toUnitPulse();

virtual IPulse* operator+(IPulse&) ;
virtual bool operator==(IPulse&) ;
virtual IPulse* open();
virtual std::vector<IPulse*> getvalues();
virtual std::vector<IUnit> getVariables();
virtual std::string print();
virtual bool isNegative();
};

class UnitPulse : public virtual IPulse, public virtual IUnit
{
public:
    UnitPulse(State = TRUE, int);
    Graph::UnitPulse* toUnitPulse();
    IPulse* operator+(IPulse&) ;
    IPulse* open();
    bool operator==(IPulse&) ;
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
    std::string print();
    bool isNegative();
};

class Pulse : public virtual IPulse
{
    std::vector<IPulse*> pulses;
    bool const negative;
public:
    Pulse(bool = true);
    Pulse(bool = true, std::vector<IPulse*>);
    Graph::Pulse* toPulse();
    IPulse* open();
    IPulse* operator+(IPulse&);
    bool operator==(IPulse&);
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
    std::string print();
    bool isNegative();
};

class AggregatedPulse : public virtual IPulse
{
    const bool negative;
    long int size;
    std::vector<std::vector<IUnit>> exclusionSet;
    std::vector<IPulse*> definitiveSet;
public:
    AggregatedPulse(bool = false);
    AggregatedPulse(Graph::Pulse&, Graph::Pulse&);
    Graph::AggregatedPulse* toAggregatedPulse();
    IPulse* open();
    IPulse* operator+(IPulse&) ;
    bool operator==(IPulse&);
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
};

```

```

        std::string print();
        bool isNegative();
};

class Node
{
public:
    void virtual propogate(Utilities::Stack, int);

    Node(int);

    const int me;
};

class NVariable : public virtual Node
{
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, int);

    NVariable(int, Accumulator*);
};

class NAnd : public virtual Node
{
    Node* left;
    Node* right;
public:
    void propagate(Utilities::Stack, int);

    NAnd(int, Node*, Node*);
};

class NNot : public virtual Node
{
    Node* fanin;
public:
    void propagate(Utilities::Stack, int);

    NNot(int, Node*);
};

class NLatch : public virtual Node
{
    Node* fanin;
    const int next_state;
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, int);

    NLatch(int, int, Node*, Accumulator*);
};

class NTrue : public virtual Node
{
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, int);

    NTrue(Accumulator*);
};

class NFalse : public virtual Node
{

```

```

        Accumulator* accumulator;
public:
    void propagate(Uilities::Stack, int);

    NFalse(Accumulator*);
};

class NodeHandler : public virtual Node
{
    Node* node;
public:
    void propagate(Uilities::Stack, int);

    NodeHandler();

    NodeHandler(Node*);

    void set_node(Node*);
};

class Propagate
{
    Uilities::Stack sat_trace;
    int last_node;
    Node* target;

public:
    Propagate(Uilities::Stack, int, Node*);

    void operator()();
};

class IStartPoint
{
public:
    void virtual start();
};

class StartPoint : public IStartPoint
{
    Node** fanins;
    const int size;
public:
    StartPoint(Node** nodes, int size);
    StartPoint(Node**);

    void start();
};

class Accumulator
{
    static struct AccNode
    {
        std::mutex mutex;
        bool master;
        std::vector<IPulse*> signals;
        std::vector<AccNode*> nodes;

        AccNode(bool);
        void add_pulse(UnitPulse*);
        void add_node(AccNode*);
        IPulse* fold();
    };
};

```

```

Utilities::Stack::Entry* master_pointer;
std::mutex mutex;
std::map<Utilities::Stack::Entry*, AccNode*> accNodes;

AccNode* get_node(Utilities::Stack::Entry*);
void add_node(Utilities::Stack::Entry*, AccNode*);
public:
    Accumulator();
    void accumulate(Utilities::Stack, int, bool);
    IPulse* solve();
    IPulse* solve(std::map<int, bool>&);

    class Add
    {
        std::mutex mutex;
        std::condition_variable cv;
        const int number_of_operands;
        std::vector<IPulse*> operands;
        IPulse** result_buffer;
        Add* adder;

    public:
        Add(int, Add* = NULL, IPulse* = NULL);

        void add_operand(IPulse*);

        void operator()();
    };

    static class Fold
    {
        Add* adder;
        AccNode* node;
    public:
        Fold(Add*, AccNode*);

        void operator()();
    };
};
}

```

Final:

```

#pragma once
#include "utilities.h"

namespace Graph
{
    enum State
    {
        ANY, TRUE, FALSE, CONFLICT
    };

    struct IUnit
    {
        const State value;
        const int variable;
        IUnit(State, int);
        IUnit(IUnit*);
        IUnit& operator=(IUnit&);
    };
}

```

```

struct IPulse
{
    static enum PulseType {
        UnitPulse, Pulse, AggregatedPulse
    };

    const PulseType type;

    IPulse(PulseType);

    virtual IPulse* operator+(IPulse&);
    virtual bool operator==(IPulse&);
    virtual IPulse* open();
    virtual std::vector<IPulse*> getvalues();
    virtual std::vector<IUnit> getVariables();
    virtual std::string print();
    virtual bool isNegative();
};

class UnitPulse : public virtual IPulse, public virtual IUnit
{
public:
    UnitPulse(State, int);
    IPulse* operator+(IPulse&);
    IPulse* open();
    bool operator==(IPulse&);
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
    std::string print();
    bool isNegative();
};

class Pulse : public virtual IPulse
{
    std::vector<IPulse*> pulses;
    bool const negative;
public:
    Pulse(bool = true);
    Pulse(bool = true, std::vector<IPulse*> =
std::vector<IPulse*>());
    IPulse* open();
    IPulse* operator+(IPulse&);
    bool operator==(IPulse&);
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
    std::string print();
    bool isNegative();
};

class AggregatedPulse : public virtual IPulse
{
    int size;
    std::vector<std::vector<IUnit>> exclusionSet;
    std::vector<IPulse*> definitiveSet;
public:
    AggregatedPulse();
    AggregatedPulse(Graph::Pulse&, Graph::Pulse&);
    IPulse* open();
    IPulse* operator+(IPulse&);
    bool operator==(IPulse&);
    std::vector<IPulse*> getvalues();
    std::vector<IUnit> getVariables();
    std::string print();
};

```



```

        bool isNegative();
};

class Accumulator
{
    static struct AccNode
    {
        std::mutex mutex;
        bool master;
        std::vector<IPulse*> signals;
        IPulse* intermediate_value;
        AccNode* next_node;
        int child_nodes;
        int fold_count;

        AccNode();
        void add_child();
        void set_master();
        void ripe(IPulse&);
        void add_pulse(UnitPulse*);
        void add_node(AccNode*);
        void fold();
        IPulse* get_result();
    };

    Utilities::Stack::Entry* master_pointer;
    std::mutex mutex;
    std::map<Utilities::Stack::Entry*, AccNode*> accNodes;

    AccNode* get_node(Utilities::Stack::Entry*);
    void add_node(Utilities::Stack::Entry*, AccNode*);
public:
    Accumulator();
    void accumulate(Utilities::Stack, int, bool);
    IPulse* solve();
    IPulse* solve(std::map<int, bool>&);

    static class Fold
    {
    {
        AccNode* node;
    public:
        Fold(AccNode*);

        void operator()();
    };
};

class Node
{
public:
    void virtual propagate(Utilities::Stack, Utilities::Stack);

    Node(int);

    const int me;
};

class NVariable : public virtual Node
{
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NVariable(int, Accumulator*);
};

```

```

};

class NAnd : public Node
{
    Node* left;
    Node* right;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NAnd(int, Node*, Node*);
};

class NNot : public virtual Node
{
    Node* fanin;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NNot(int, Node*);
};

class NLatch : public virtual Node
{
    Node* fanin;
    const int next_state;
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NLatch(int, int, Node*, Accumulator*);
};

class NTrue : public virtual Node
{
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NTrue(Accumulator*);
};

class NFalse : public virtual Node
{
    Accumulator* accumulator;
public:
    void propagate(Utilities::Stack, Utilities::Stack);

    NFalse(Accumulator*);
};

class NodeHandler : public virtual Node
{
public:
    Node* node;

    void propagate(Utilities::Stack, Utilities::Stack);

    NodeHandler();

    NodeHandler(Node*);

    void set_node(Node*);
};

```

```

class Propagate
{
    Utilities::Stack sat_trace;
    Utilities::Stack latch_trace;
    Node* target;

public:
    Propagate(Utilities::Stack, Utilities::Stack, Node*);

    void operator()();
};

class IStartPoint
{
public:
    void virtual start();
};

class StartPoint : public virtual IStartPoint
{
    Node** fanins;
    const int size;

public:
    StartPoint(Node** nodes, int size);
    StartPoint(Node**);

    void start();
};
}

```

UnitPulse.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    IUnit::IUnit(State value, int variable) : value(value),
    variable(variable) {}

    UnitPulse::UnitPulse(State value, int variable) :
    IPulse(PulseType::UnitPulse), IUnit(value, variable) {}

    IPulse* UnitPulse::operator+(IPulse& pulse)
    {
        switch (pulse.type)
        {
            case PulseType::UnitPulse:
                UnitPulse* upulse = pulse.toUnitPulse();
                if (variable != upulse->variable)
                {
                    if (upulse -> value != State::CONFLICT)
                    {
                        std::vector<IPulse*> pulses;
                        pulses.push_back(this);
                        pulses.push_back(&pulse);
                        return new Graph::Pulse(false, pulses);
                    }
                }
            else
            {
                return &pulse;
            }
        }
    }
}

```

```

        }
    }
    else if (value == upulse->value || upulse->value ==
State::ANY)
    {
        return this;
    }
    else if (value == State::ANY)
    {
        return &pulse;
    }
    else
    {
        return new UnitPulse(CONFLICT, 0);
    }
default:
    return pulse + *this;
}

}

Graph::UnitPulse* UnitPulse::toUnitPulse()
{
    return this;
}

IPulse* UnitPulse::open()
{
    return new UnitPulse(ANY, variable);
}

bool UnitPulse::operator==(IPulse& lPulse)
{
    for (IUnit& unit : lPulse.getVariables())
    {
        if (this->variable == unit.variable)
        {
            return true;
        }
    }
    return false;
}

std::vector<IUnit> UnitPulse::getVariables()
{
    std::vector<IUnit> vec;
    vec.push_back(*this);
    return vec;
}

std::vector<IPulse*> UnitPulse::getvalues()
{
    std::vector<IPulse*> vec;
    vec.push_back(this);
    return vec;
}

std::string UnitPulse::print()
{
    return this->variable + " = " + this->value;
}

bool UnitPulse::isNegative() { return this->value == FALSE; }

```

```
}
```

Final:

```
#include "Alexandra.h"
```

```
namespace Graph
```

```
{
```

```
    IUnit::IUnit(State value, int variable) : value(value),  
variable(variable) {}
```

```
    IUnit::IUnit(IUnit* unit) : value(unit->value), variable(unit->variable)  
{}
```

```
    IUnit& IUnit::operator=(IUnit& unit)  
    {  
        return *new IUnit(unit);  
    }
```

```
    UnitPulse::UnitPulse(State value, int variable) :  
IPulse(PulseType::UnitPulse), IUnit(value,variable) {}
```

```
    IPulse* UnitPulse::operator+(IPulse& pulse)  
    {  
        switch (pulse.type)  
        {  
            case PulseType::UnitPulse:  
            {  
                UnitPulse* upulse =  
dynamic_cast<Graph::UnitPulse*>(&pulse);  
                if (variable != upulse->variable)  
                {  
                    if (upulse->value != State::CONFLICT)  
                    {  
                        std::vector<IPulse*> pulses;  
                        pulses.push_back(this);  
                        pulses.push_back(&pulse);  
                        return new Graph::Pulse(false, pulses);  
                    }  
                    else  
                    {  
                        return &pulse;  
                    }  
                }  
            }  
        }  
    }
```

```
    }  
    else if (value == upulse->value || upulse->value ==  
State::ANY)  
    {  
        return this;  
    }  
    else if (value == State::ANY)  
    {  
        return &pulse;  
    }  
    else  
    {  
        return new UnitPulse(CONFLICT, 0);  
    }  
}  
default:  
    return pulse + *this;
```

```

    }
}

IPulse* UnitPulse::open()
{
    return new UnitPulse(ANY, variable);
}

bool UnitPulse::operator==(IPulse& lPulse)
{
    for (IUnit unit : lPulse.getVariables())
    {
        if (this->variable == unit.variable)
        {
            return true;
        }
    }
    return false;
}

std::vector<IUnit> UnitPulse::getVariables()
{
    std::vector<IUnit> vec;
    vec.push_back(IUnit(value, variable));
    return vec;
}

std::vector<IPulse*> UnitPulse::getvalues()
{
    std::vector<IPulse*> vec;
    vec.push_back(this);
    return vec;
}

std::string UnitPulse::print()
{
    std::string result = "";
    result += std::to_string(variable);
    result += " = ";
    switch (value)
    {
        case TRUE:
            result += "TRUE";
            break;
        case FALSE:
            result += "FALSE";
            break;
        case CONFLICT:
            result += "CONFLICT";
            break;
        case ANY:
            result += "ANY";
    }
    return result;
}

bool UnitPulse::isNegative() { return false; }
}

```

Stack.cpp

Original:

```
#include "Alexandra.h"

namespace Utilities
{
    Stack::Entry::Entry(int value, Entry* prev) : value(value), prev(prev)
    {}

    Stack::Entry::Entry(int value) : value(value), prev(NULL) {}

    Stack::Entry::Entry(Entry* entry) : value(entry->value), prev(entry-
>prev) {}

    Stack::Stack() : entry(new Entry(0)) {}

    Stack::Stack(Stack* stack) : entry(stack->top_entry()) {}

    Stack& Stack::operator=(Stack& stack)
    {
        return new Stack(stack);
    }

    int Stack::top()
    {
        return entry->value;
    }

    Stack::Entry* Stack::top_entry()
    {
        return entry;
    }

    void Stack::push(int value)
    {
        Stack::Entry* new_entry = new Entry(value, entry);
        entry = new_entry;
    }

    Stack::Entry* Stack::pop()
    {
        Stack::Entry* new_entry = entry;
        entry = entry->prev;
        return new_entry;
    }

    bool Stack::empty()
    {
        return entry == NULL;
    }
}
```

Final:

```
#include "Alexandra.h"

namespace Utilities
{
```

```

    Stack::Entry::Entry(int value, Entry* prev) : value(value), prev(prev)
{}

    Stack::Entry::Entry(int value) : value(value), prev(NULL) {}

    Stack::Entry::Entry(Entry* entry) : value(entry->value), prev(entry-
>prev) {}

    Stack::Stack() : entry(new Entry(0)) {}

    Stack::Stack(Stack* stack) : entry(stack->top_entry()) {}

    Stack& Stack::operator=(Stack& stack)
    {
        return *new Stack(stack);
    }

    int Stack::top()
    {
        return entry->value;
    }

    Stack::Entry* Stack::top_entry()
    {
        return entry;
    }

    void Stack::push(int value)
    {
        Stack::Entry* new_entry = new Entry(value, entry);
        entry = new_entry;
    }

    Stack::Entry* Stack::pop()
    {
        Stack::Entry* new_entry = entry;
        entry = entry->prev;
        return new_entry;
    }

    bool Stack::empty()
    {
        return entry == NULL;
    }
}

```

solver.h

Original:

```

#pragma once
#include "graph.h"

namespace Alexandra
{
    class Solver
    {
        Graph::Accumulator accumulator;

        Graph::IStartPoint* start_point;
    }
}

```



```

        std::map<int, Graph::NodeHandler&> nodes;

        std::ifstream file;

        const char delimiter;

        int number_of_inputs;

        int number_of_latches;

        int number_of_outputs;

        int number_of_and;

        bool validate_header();

        void add_node(int);

        void inputs();

        void latches();

        void outputs();

        void ands();

    public:

        Solver(std::string&);

        Result& solve();

        Result& solve(std::map<int, bool>);

};

class Result
{
    Graph::IPulse* value;
public:
    Result(Graph::IPulse*);

    operator std::string();

};
}

```

Final:

```

#pragma once
#include "graph.h"

namespace Alexandra
{
    class Result
    {
    public:
        Graph::IPulse* value;

        Result(Graph::IPulse*);

        operator std::string();

    };
}

```

```

class Solver
{
    Graph::Accumulator accumulator;

    Graph::IStartPoint* start_point;

    std::map<int, Graph::NodeHandler*> nodes;

    std::ifstream file;

    const char delimiter;

    int number_of_inputs;

    int number_of_latches;

    int number_of_outputs;

    int number_of_and;

    bool validate_header();

    void add_node(int);

    void inputs();

    void latches();

    void outputs();

    void ands();

public:

    Solver(std::string&);

    Result& solve();

    Result& solve(std::map<int, bool>);

};
}

```

Pulse.cpp

Original:

```
#include "Alexandra.h"
```

```

namespace Graph
{
    bool Pulse::operator==(IPulse& pulse)
    {
        std::vector<IUnit> target_variables = pulse.getVariables();
        std::vector<IUnit> variables = getVariables();
        for (IUnit& unit : variables)
        {
            for (IUnit& target_unit : target_variables)
            {
                if (unit.variable == target_unit.variable)
                {
                    return true;
                }
            }
        }
    }
}

```

```

    }
    }
    }
    return false;
}

IPulse* Pulse::operator+(IPulse& pulse)
{
    switch (pulse.type)
    {
    case PulseType::AggregatedPulse:
        return pulse + * this;
    case PulseType::UnitPulse:
    {
        std::vector<IPulse*> old_buffer, new_buffer;
        bool satisfied = false;
        for (IPulse* p : pulses)
        {
            if (*p == pulse)
            {
                IPulse* intermediate = *p + pulse;
                if (*intermediate ==
Graph::UnitPulse(CONFLICT, 0))
                {
                    if (negative)
                    {
                        satisfied = true;
                    }
                    else
                    {
                        return intermediate;
                    }
                }
                else
                {
                    new_buffer.push_back(intermediate);
                }
            }
            else
            {
                old_buffer.push_back(p);
            }
        }
        if (negative)
        {
            if (satisfied)
            {
                for (IPulse* p : old_buffer)
                {
                    new_buffer.push_back(p->open());
                }
            }
            else
            {
                if (new_buffer.size() == 0)
                {
                    new_buffer.push_back(&pulse);
                }
                new_buffer.push_back(new Pulse(true,
old_buffer));
            }
        }
    }
}

```

```

        return new Pulse(false, new_buffer);
    }
    else
    {
        if (new_buffer.size() == 0)
        {
            new_buffer.push_back(&pulse);
        }
        for (IPulse* p : new_buffer)
        {
            if (p->type == PulseType::UnitPulse)
            {
                old_buffer.push_back(p);
            }
            else if (p->type == PulseType::Pulse
&& !p->isNegative())
            {
                std::vector<IPulse*> values = p-
>getvalues();
                for (IPulse* value : values)
                {
                    old_buffer.push_back(value);
                }
            }
            else
            {
                old_buffer.push_back(p);
            }
        }
        return new Pulse(false, old_buffer);
    }
}
case PulseType::Pulse:
{
    Pulse* ppulse = pulse.toPulse();
    if (ppulse->negative == true && negative == true)
    {
        std::vector<IPulse*> common_buffer, left_buffer,
right_buffer, new_buffer;
        for (IPulse* p : pulses)
        {
            if (*p == pulse)
            {
                common_buffer.push_back(p);
            }
            else
            {
                left_buffer.push_back(p);
            }
        }
        for (IPulse* rp : ppulse->pulses)
        {
            bool present = false;
            for (IPulse* lp : common_buffer)
            {
                if (*lp == *rp)
                {
                    present = true;
                    IPulse* intermediate =
(*lp + *rp);
                    Graph::UnitPulse(CONFLICT, 0))
                {

```

```

intermediate;
Graph::AggregatedPulse(*this, *ppulse);
}
else
{
    new_buffer.push_back(intermediate);
    break;
}
}
}
if (!present)
{
    right_buffer.push_back(rp);
}
}
for (IPulse* p : left_buffer)
{
    new_buffer.push_back(p);
}
for (IPulse* p : right_buffer)
{
    new_buffer.push_back(p);
}
return new Pulse(true, new_buffer);
}
else if (ppulse->negative == false && negative == false)
{
    std::vector<IPulse*> left_buffer, right_buffer,
unit_buffer, pulse_buffer;
    std::vector<IPulse*> first_pointer_buffer,
second_pointer_buffer;
    for (IPulse* lp : pulses)
    {
        bool equivalent = false;
        for (IPulse* rp : ppulse->pulses)
        {
            if (*lp == *rp)
            {
                equivalent = true;
                IPulse* intermediate =
                    *lp + *rp;
                Graph::UnitPulse(CONFLICT, 0))
                {
                    return
                }
            }
            else
            {
                if
                {
                    unit_buffer.push_back(intermediate);
                }
            }
            else
            {
                first_pointer_buffer.push_back(intermediate);
                second_pointer_buffer.push_back(intermediate);
            }
        }
    }
}

```

```

    }
    }
    }
    if (!equivalent)
    {
        left_buffer.push_back(lp);
    }
}
for (IPulse* p : ppulse->pulses)
{
    if (*p != *this)
    {
        right_buffer.push_back(p);
    }
}
for (int i = 0; i < first_pointer_buffer.size();
i++)
{
    pulse_buffer.push_back(first_pointer_buffer[i]);
    for (int j = i + 1; j <
first_pointer_buffer.size(); j++)
    {
        if (second_pointer_buffer[j] !=
NULL)
        {
            IPulse* temp =
            if
            {
                pulse_buffer.back();
                (*second_pointer_buffer[j] == *temp)
                {
                    pulse_buffer.pop_back();
                    pulse_buffer.push_back(*second_pointer_buffer[j] + *temp);
                    delete temp;
                    second_pointer_buffer[j] = NULL;
                }
            }
        }
        for (IPulse* p : second_pointer_buffer)
        {
            if (p != NULL)
            {
                pulse_buffer.push_back(p);
            }
        }
    }
    std::vector<IPulse*> new_buffer;
    for (IPulse* u : unit_buffer)
    {
        new_buffer.push_back(u);
    }
    for (IPulse* p : pulse_buffer)
    {
        if (p->type == PulseType::UnitPulse)
        {
            return p;
        }
        else if (p->type ==
PulseType::AggregatedPulse)

```

```

        {
            new_buffer.push_back(p);
        }
        else
        {
            if (p->isNegative())
            {
                new_buffer.push_back(p);
            }
            else
            {
                for (IPulse* value : p-
>toPulse()->getvalues())
                {
                    new_buffer.push_back(value);
                }
            }
        }
    }
    return new Pulse(false, new_buffer);
}
else if (ppulse->negative == true && negative == false)
{
    if (*this != pulse)
    {
        std::vector<IPulse*> new_buffer;
        for (IPulse* p : pulses)
        {
            new_buffer.push_back(p);
        }
        new_buffer.push_back(&pulse);
        return new Pulse(false, new_buffer);
    }

    bool satisfied = false;
    std::vector<IPulse*> intermediate_buffer,
new_buffer;

    for (IPulse* lp : pulses)
    {
        std::vector<IPulse*> equivalence_buffer;
        for (IPulse* rp : ppulse->pulses)
        {
            if (*lp == *rp)
            {
                equivalence_buffer.push_back(rp);
            }
        }
        IPulse* equivalent_pulse;
        if (equivalence_buffer.size() > 0)
        {
            if (equivalence_buffer.size() >
1)
            {
                equivalent_pulse = new
Pulse(true, equivalence_buffer);
            }
            else
            {
                equivalent_pulse =
equivalence_buffer.back();
            }
        }
    }
}

```

```

+ *equivalent_pulse;
Graph::UnitPulse(CONFLICT, 0))

intermediate_buffer.push_back(lp);

intermediate_pulse;

intermediate_buffer.push_back(intermediate_pulse);

delete equivalent_pulse;

}
else
{
    new_buffer.push_back(lp);
}
}
bool equivalent = true;
std::vector<IPulse*> remaining_buffer;
for (IPulse* p : ppulse->pulses)
{
    if (*p != *this)
    {
        equivalent = false;
        remaining_buffer.push_back(p);
    }
}
if ((new_buffer.empty() || equivalent == true)
&& satisfied == false)
{
    return new Graph::UnitPulse(CONFLICT,
0);
}
else
{
    if (!remaining_buffer.empty())
    {
        IPulse* remaining = new
Pulse(true, remaining_buffer);

intermediate_buffer.push_back(remaining->open());

delete remaining;
    }
    else
    {
        intermediate_buffer.push_back(remaining);
    }
}
for (IPulse* p : intermediate_buffer)
{
    if (p->type == PulseType::Pulse
&& !p->isNegative())
    {
        for (IPulse* value : p-
>getvalues())

```



```

        {
new_buffer.push_back(value);
        }
    }
    else
    {
        new_buffer.push_back(p);
    }
}
return new Pulse(false, new_buffer);
}

}
else
{
    return pulse + *this;
}
}
}

Pulse::Pulse(bool negative) : IPulse(PulseType::Pulse),
negative(negative), pulses() {}

Pulse::Pulse(bool negative, std::vector<IPulse*> pulses) :
IPulse(PulseType::Pulse), negative(negative), pulses(pulses){}

Graph::Pulse* Pulse::toPulse()
{
    return this;
}

IPulse* Pulse::open()
{
    std::vector<IPulse*> result;
    for (IUnit u : getVariables())
    {
        result.push_back(new Graph::UnitPulse(ANY, u.variable));
    }
    return new Pulse(false, result);
}

std::vector<IPulse*> Pulse::getvalues()
{
    return pulses;
}

std::vector<IUnit> Pulse::getVariables()
{
    std::vector<IUnit> vec;
    for (IPulse* pulse : pulses)
    {
        for (IUnit unit : pulse->getVariables())
        {
            vec.push_back(unit);
        }
    }
    return vec;
}

std::string Pulse::print()

```

```

{
    std::string result;
    if (negative)
    {
        result = "Not ";
    }

    for (int i = 0; i < pulses.size(); i++)
    {
        result += pulses.at(i)->print();

        if (i < pulses.size() - 1)
        {
            result += ", ";
        }
    }

    if (negative)
    {
        result += ")";
    }

    return result;
}

bool Pulse::isNegative()
{
    return negative;
}
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{
    bool Pulse::operator==(IPulse& pulse)
    {
        std::vector<IUnit> target_variables = pulse.getVariables();
        std::vector<IUnit> variables = getVariables();
        for (IUnit unit : variables)
        {
            for (IUnit target_unit : target_variables)
            {
                if (unit.variable == target_unit.variable)
                {
                    return true;
                }
            }
        }
        return false;
    }

    IPulse* Pulse::operator+(IPulse& pulse)
    {
        switch (pulse.type)
        {
            case PulseType::AggregatedPulse:
                return pulse + *this;
            case PulseType::UnitPulse:

```

```

{
    std::vector<IPulse*> old_buffer, new_buffer;
    bool satisfied = false;
    for (IPulse* p : pulses)
    {
        if (*p == pulse)
        {
            IPulse* intermediate = *p + pulse;
            if (*intermediate ==
Graph::UnitPulse(CONFLICT, 0))
            {
                if (negative)
                {
                    satisfied = true;
                }
                else
                {
                    return intermediate;
                }
            }
            else
            {
                new_buffer.push_back(intermediate);
            }
        }
        else
        {
            old_buffer.push_back(p);
        }
    }
    if (negative)
    {
        if (satisfied)
        {
            for (IPulse* p : old_buffer)
            {
                new_buffer.push_back(p->open());
            }
        }
        else
        {
            if (new_buffer.size() == 0)
            {
                new_buffer.push_back(&pulse);
            }
            new_buffer.push_back(new Pulse(true,
old_buffer));
        }
        return new Pulse(false, new_buffer);
    }
    else
    {
        if (new_buffer.size() == 0)
        {
            new_buffer.push_back(&pulse);
        }
        for (IPulse* p : new_buffer)
        {
            if (p->type == PulseType::UnitPulse)
            {
                old_buffer.push_back(p);
            }
        }
    }
}

```

```

    }
    else if (p->type == PulseType::Pulse
&& !p->isNegative())
    {
        std::vector<IPulse*> values = p-
>getvalues();
        for (IPulse* value : values)
        {
            old_buffer.push_back(value);
        }
    }
    else
    {
        old_buffer.push_back(p);
    }
}
return new Pulse(false, old_buffer);
}
}
case PulseType::Pulse:
{
    Pulse* ppulse = dynamic_cast<Graph::Pulse*>(&pulse);
    if (ppulse->negative == true && negative == true)
    {
        std::vector<IPulse*> common_buffer, left_buffer,
right_buffer, new_buffer;
        for (IPulse* p : pulses)
        {
            if (*p == pulse)
            {
                common_buffer.push_back(p);
            }
            else
            {
                left_buffer.push_back(p);
            }
        }
        for (IPulse* rp : ppulse->pulses)
        {
            bool present = false;
            for (IPulse* lp : common_buffer)
            {
                if (*lp == *rp)
                {
                    present = true;
                    IPulse* intermediate =
(*lp + *rp);
                    Graph::UnitPulse(CONFLICT, 0)
intermediate;
                    Graph::AggregatedPulse(*this, *ppulse);
                    new_buffer.push_back(intermediate);
                    break;
                }
            }
        }
    }
}
}

```

```

        if (!present)
        {
            right_buffer.push_back(rp);
        }
    }
    for (IPulse* p : left_buffer)
    {
        new_buffer.push_back(p);
    }
    for (IPulse* p : right_buffer)
    {
        new_buffer.push_back(p);
    }
    return new Pulse(true, new_buffer);
}
else if (ppulse->negative == false && negative == false)
{
    std::vector<IPulse*> left_buffer, right_buffer,
unit_buffer, pulse_buffer;
    std::vector<IPulse*> first_pointer_buffer,
second_pointer_buffer;
    for (IPulse* lp : pulses)
    {
        bool equivalent = false;
        for (IPulse* rp : ppulse->pulses)
        {
            if (*lp == *rp)
            {
                equivalent = true;
                IPulse* intermediate =
*lp + *rp;
                Graph::UnitPulse(CONFLICT, 0)
intermediate;

                if (intermediate->type == PulseType::UnitPulse)
                {
                    unit_buffer.push_back(intermediate);

                    first_pointer_buffer.push_back(intermediate);
                    second_pointer_buffer.push_back(intermediate);
                }
            }
        }
        if (!equivalent)
        {
            left_buffer.push_back(lp);
        }
    }
    for (IPulse* p : ppulse->pulses)
    {
        if (*p != *this)
        {

```

```

        right_buffer.push_back(p);
    }
    }
    for (int i = 0; i < first_pointer_buffer.size();
i++)
    {
        pulse_buffer.push_back(first_pointer_buffer[i]);
        for (int j = i + 1; j <
first_pointer_buffer.size(); j++)
        {
            if (second_pointer_buffer[j] !=
NULL)
            {
                IPulse* temp =
                if
                {
                    pulse_buffer.back();
                    (*second_pointer_buffer[j] == *temp)

                    pulse_buffer.pop_back();
                    pulse_buffer.push_back(*second_pointer_buffer[j] + *temp);
                                                                    //delete temp;
                    second_pointer_buffer[j] = NULL;
                }
            }
        }
        for (IPulse* p : second_pointer_buffer)
        {
            if (p != NULL)
            {
                pulse_buffer.push_back(p);
            }
        }
    }
    std::vector<IPulse*> new_buffer;
    for (IPulse* u : unit_buffer)
    {
        new_buffer.push_back(u);
    }
    for (IPulse* p : pulse_buffer)
    {
        if (p->type == PulseType::UnitPulse)
        {
            return p;
        }
        else if (p->type ==
PulseType::AggregatedPulse)
        {
            new_buffer.push_back(p);
        }
        else
        {
            if (p->isNegative())
            {
                new_buffer.push_back(p);
            }
            else
            {
                for (IPulse* value :
dynamic_cast<Graph::Pulse*>(p)->getvalues())

```

```

        {
new_buffer.push_back(value);
        }
    }
}
return new Pulse(false, new_buffer);
}
else if (ppulse->negative == true && negative == false)
{
    if (*this != pulse)
    {
        std::vector<IPulse*> new_buffer;
        for (IPulse* p : pulses)
        {
            new_buffer.push_back(p);
        }
        new_buffer.push_back(&pulse);
        return new Pulse(false, new_buffer);
    }

    bool satisfied = false;
    std::vector<IPulse*> intermediate_buffer,
new_buffer;

    for (IPulse* lp : pulses)
    {
        std::vector<IPulse*> equivalence_buffer;
        for (IPulse* rp : ppulse->pulses)
        {
            if (*lp == *rp)
            {
                equivalence_buffer.push_back(rp);
            }
        }
        IPulse* equivalent_pulse;
        if (equivalence_buffer.size() > 0)
        {
            if (equivalence_buffer.size() >
1)
            {
                equivalent_pulse = new
Pulse(true, equivalence_buffer);
            }
            else
            {
                equivalent_pulse =
equivalence_buffer.back();
            }
            IPulse* intermediate_pulse = *lp
+ *equivalent_pulse;
            Graph::UnitPulse(CONFLICT, 0))
            if (*intermediate_pulse ==
            {
                satisfied = true;
                //delete
intermediate_pulse;
            }
            else
            {

```

```

intermediate_buffer.push_back(intermediate_pulse);
                                }
                                //delete equivalent_pulse;
                                }
                                else
                                {
                                    new_buffer.push_back(lp);
                                }
                            }
                            bool equivalent = true;
                            std::vector<IPulse*> remaining_buffer;
                            for (IPulse* p : ppulse->pulses)
                            {
                                if (*p != *this)
                                {
                                    equivalent = false;
                                    remaining_buffer.push_back(p);
                                }
                            }
                            if ((new_buffer.empty() || equivalent == true)
&& satisfied == false)
                            {
                                return new Graph::UnitPulse(CONFLICT,
0);
                            }
                            else
                            {
                                if (!remaining_buffer.empty())
                                {
                                    IPulse* remaining = new
Pulse(true, remaining_buffer);

                                    intermediate_buffer.push_back(remaining->open());
                                                                    //delete remaining;
                                }
                                else
                                {
                                    intermediate_buffer.push_back(remaining);
                                }
                            }
                            for (IPulse* p : intermediate_buffer)
                            {
                                if (p->type == PulseType::Pulse
&& !p->isNegative())
                                {
                                    for (IPulse* value : p-
>getvalues())
                                    {
                                        new_buffer.push_back(value);
                                    }
                                }
                                else
                                {
                                    new_buffer.push_back(p);
                                }
                            }
                            return new Pulse(false, new_buffer);
                        }
}

```



```

        }
        else
        {
            return pulse + *this;
        }
    }
}

Pulse::Pulse(bool negative) : IPulse(PulseType::Pulse),
negative(negative), pulses() {}

Pulse::Pulse(bool negative, std::vector<IPulse*> pulses) :
IPulse(PulseType::Pulse), negative(negative), pulses(pulses) {}

IPulse* Pulse::open()
{
    std::vector<IPulse*> result;
    for (IUnit u : getVariables())
    {
        result.push_back(new Graph::UnitPulse(ANY, u.variable));
    }
    return new Pulse(false, result);
}

std::vector<IPulse*> Pulse::getvalues()
{
    return pulses;
}

std::vector<IUnit> Pulse::getVariables()
{
    std::vector<IUnit> vec;
    for (IPulse* pulse : pulses)
    {
        for (IUnit unit : pulse->getVariables())
        {
            vec.push_back(unit);
        }
    }
    return vec;
}

std::string Pulse::print()
{
    std::string result;
    if (negative)
    {
        result = "Not ";
    }

    for (int i = 0; i < pulses.size(); i++)
    {
        result += pulses.at(i)->print();

        if (i < pulses.size() - 1)
        {
            result += ", ";
        }
    }

    if (negative)
    {

```

```

        result += ")";
    }

    return result;
}

bool Pulse::isNegative()
{
    return negative;
}
}

```

NTrue.cpp

Original:

```

#include "graph.h"

namespace Graph
{
    NTrue::NTrue(Accumulator* accumulator) : Node(1),
    accumulator(accumulator) {}

    void NTrue::propagate(Utilities::Stack sat_trace, int last_node)
    {
        accumulator->accumulate(sat_trace, me);
    }
}

```

Final:

```

#include "graph.h"

namespace Graph
{
    NTrue::NTrue(Accumulator* accumulator) : Node(1),
    accumulator(accumulator) {}

    void NTrue::propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace)
    {
        accumulator->accumulate(sat_trace, me, true);
    }
}

```

NFalse.cpp

Original:

```

#include "graph.h"

namespace Graph
{
    NFalse::NFalse(Accumulator* accumulator) : Node(0),
    accumulator(accumulator) {}
}

```

```

        void NFalse::propagate(Utilities::Stack sat_trace, int last_node)
        {
            accumulator->accumulate(sat_trace, me);
        }
    }
}

```

Final:

```

#include "graph.h"

namespace Graph
{
    NFalse::NFalse(Accumulator* accumulator) : Node(0),
    accumulator(accumulator) {}

    void NFalse::propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace)
    {
        accumulator->accumulate(sat_trace, me, false);
    }
}

```

IPulse.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    IPulse::IPulse(PulseType type) : type(type) {}

    Graph::AggregatedPulse* IPulse::toAggregatedPulse() {}

    Graph::Pulse* IPulse::toPulse() {}

    Graph::UnitPulse* IPulse::toUnitPulse() {}

    Graph::IPulse* IPulse::open() {}

    bool IPulse::operator==(IPulse&){}

    IPulse* IPulse::operator+(IPulse&) {}

    std::vector <IPulse*> IPulse::getvalues() {}

    std::vector<IUnit> IPulse::getVariables() {}

    std::string IPulse::print() {}

    bool IPulse::isNegative() {}
}

```

Final:

```

#include "Alexandra.h"

```

```

namespace Graph
{
    IPulse::IPulse(PulseType type) : type(type) {}

    Graph::IPulse* IPulse::open() { return NULL; }

    bool IPulse::operator==(IPulse&) { return false; };

    IPulse* IPulse::operator+(IPulse&) { return NULL; }

    std::vector<IPulse*> IPulse::getvalues() { return
std::vector<IPulse*>(); }

    std::vector<IUnit> IPulse::getVariables() { return std::vector<IUnit>(); }

    std::string IPulse::print() { return ""; }

    bool IPulse::isNegative() { return false; }
}

```

AggregatedPulse.cpp

Original:

```
#include "Alexandra.h"
```

```

namespace Graph
{
    AggregatedPulse::AggregatedPulse(bool negative) :
    IPulse(PulseType::AggregatedPulse), negative(negative), exclusionSet(),
    definitiveSet() {}

    AggregatedPulse::AggregatedPulse(Graph::Pulse& left, Graph::Pulse&
right) : IPulse(PulseType::AggregatedPulse), negative(false), exclusionSet(),
definitiveSet()
    {
        if (left.isNegative() && right.isNegative())
        {
            std::vector<int> common_variables, left_variables,
right_variables;

            std::vector<IPulse*> left_buffer = left.getvalues();
            std::vector<IPulse*> right_buffer = right.getvalues();
            std::vector<IUnit> left_exclusive = left.getVariables();
            std::vector<IUnit> right_exclusive =
right.getVariables();

            for (IPulse* lp : left_buffer)
            {
                bool common = false;
                for (IPulse* rp : right_buffer)
                {
                    if (*lp == *rp)
                    {
                        common = true;
                        common_variables.push_back(lp-
>toUnitPulse()->variable);
                    }
                }
                if (!common)

```

```

        {
            left_variables.push_back(lp-
>toUnitPulse()->variable);
        }
    }
    for (IPulse* rp : right_buffer)
    {
        int rv = rp->toUnitPulse()->variable;
        bool common = false;
        for (int lv : common_variables)
        {
            if (lv == rv)
            {
                common = true;
            }
        }
        if (!common)
        {
            right_variables.push_back(rv);
        }
    }

    for (int lv : left_variables)
    {
        Graph::UnitPulse* lp = new Graph::UnitPulse(ANY,
lv);

        right_exclusive.push_back(*lp);
        definitiveSet.push_back(lp);
    }
    for (int rv : right_variables)
    {
        Graph::UnitPulse* lp = new Graph::UnitPulse(ANY,
rv);

        left_exclusive.push_back(*lp);
        definitiveSet.push_back(lp);
    }
    for (int v : common_variables)
    {
        definitiveSet.push_back(new
Graph::UnitPulse(ANY, v));
    }

    exclusionSet.push_back(left_exclusive);
    exclusionSet.push_back(right_exclusive);
    size = common_variables.size() + left_variables.size() +
right_variables.size();
    }
}

Graph::AggregatedPulse* AggregatedPulse::toAggregatedPulse()
{
    return this;
}

bool AggregatedPulse::operator==(IPulse& pulse)
{
    std::vector<IUnit> target_variables = pulse.getVariables();
    std::vector<IUnit> variables = getVariables();
    for (IUnit& unit : variables)
    {
        for (IUnit& target_unit : target_variables)
        {
            if (unit.variable == target_unit.variable)

```

```

        {
            return true;
        }
    }
    return false;
}

IPulse* AggregatedPulse::operator+ (IPulse& pulse)
{
    if (pulse == Graph::UnitPulse(CONFLICT, 0))
    {
        return &pulse;
    }
    if (pulse.type == PulseType::UnitPulse || (pulse.type ==
PulseType::Pulse) && pulse.isNegative())
    {
        if (pulse != *this)
        {
            std::vector<IPulse*> buffer = pulse.getvalues();
            buffer.push_back(this);
            return new Graph::Pulse(false, buffer);
        }
        else
        {
            IPulse* prev = NULL;
            IPulse* result;
            for (std::vector<IUnit> exclude : exclusionSet)
            {
                std::vector<IPulse*> buffer;
                for (IUnit variable : exclude)
                {
                    buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
                }
                Graph::Pulse exclusion_pulse =
Graph::Pulse(true, buffer);

                if (prev == NULL)
                {
                    result = pulse +
exclusion_pulse;
                }
                else
                {
                    result = *prev +
exclusion_pulse;
                    delete prev;
                }

                if (*result ==
Graph::UnitPulse(CONFLICT, 0))
                {
                    return result;
                }
                else
                {
                    prev = result;
                }
            }
            return result;
        }
    }
    else
    {

```

```

        AggregatedPulse* result = new AggregatedPulse();
        std::vector<std::vector<IUnit>> left_buffer,
right_buffer;

        std::vector<IUnit> left_padding, right_padding;
        if (pulse.type == PulseType::Pulse)
        {
            right_buffer.push_back(pulse.getVariables());
            for (IPulse* p : pulse.getvalues())
            {
                if (*p != *this)
                {
                    right_padding.push_back(IUnit(ANY, p->toUnitPulse()->variable));
                }
            }
        }
        else
        {
            for (IPulse* exc : pulse.getvalues())
            {
                right_buffer.push_back(exc-
>getVariables());
                for (IPulse* p :
pulse.toAggregatedPulse()->definitiveSet)
                {
                    if (*p != *this)
                    {
                        right_padding.push_back(IUnit(ANY, p->toUnitPulse()->variable));
                    }
                }
            }
            for (std::vector<IUnit> vec : exclusionSet)
            {
                left_buffer.push_back(vec);
            }
            for (IPulse* p : definitiveSet)
            {
                if (*p != *this)
                {
                    left_padding.push_back(IUnit(ANY, p-
>toUnitPulse()->variable));
                }
            }

            for (std::vector<IUnit> vec : left_buffer)
            {
                vec.emplace(right_padding.begin());
                result->exclusionSet.push_back(vec);
            }
            for (std::vector<IUnit> vec : right_buffer)
            {
                vec.emplace(left_padding.begin());
                result->exclusionSet.push_back(vec);
            }
            for (IPulse* p : definitiveSet)
            {
                result->definitiveSet.push_back(p);
                result->size++;
            }
            for (IUnit u : right_padding)
            {

```

```

        result->definitiveSet.push_back(new
Graph::UnitPulse(u.value, u.variable));
        result->size++;
    }

    long int total_weight = 0;
    for (std::vector<IUnit> vec : result->exclusionSet)
    {
        long int weight = 1;
        for (IUnit u : vec)
        {
            if (u.value == ANY)
            {
                weight *= 2;
            }
        }
        total_weight += weight;
    }
    if (total_weight == pow(2, result->size))
    {
        delete result;
        return new Graph::UnitPulse(CONFLICT, 0);
    }
    else
    {
        return result;
    }
}

std::vector <IPulse*> AggregatedPulse::getvalues()
{
    std::vector<IPulse*> result;
    for (std::vector<IUnit> exclude : exclusionSet)
    {
        std::vector<IPulse*> buffer;
        for (IUnit variable : exclude)
        {
            buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
        }
        result.push_back(new Graph::Pulse(true, buffer));
    }
    return result;
}

std::vector<IUnit> AggregatedPulse::getVariables()
{
    std::vector<IUnit> vec;

    for (IPulse* pulse : definitiveSet)
    {
        for (IUnit unit : pulse->getVariables())
        {
            vec.push_back(unit);
        }
    }

    return vec;
}

IPulse* AggregatedPulse::open()
{
    return new Graph::Pulse(false, definitiveSet);
}

```



```

    }

    std::string AggregatedPulse::print()
    {
        std::string result;
        result += "Not { ";
        for (int i = 0; i < exclusionSet.size(); i++)
        {
            result += "( ";
            std::vector<IPulse*> buffer;
            for (IUnit variable : exclusionSet[i])
            {
                buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
            }
            result += Graph::Pulse(false, buffer).print();
            result += " )";
            if (i < exclusionSet.size() - 1)
            {
                result += ", ";
            }
        }
        result += " }";
        return result;
    }

    bool AggregatedPulse::isNegative()
    {
        return negative;
    }
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{
    AggregatedPulse::AggregatedPulse() : IPulse(PulseType::AggregatedPulse),
exclusionSet(), definitiveSet(), size(0) {}

    AggregatedPulse::AggregatedPulse(Graph::Pulse& left, Graph::Pulse&
right) : IPulse(PulseType::AggregatedPulse), exclusionSet(), definitiveSet(),
size(0)
    {
        if (left.isNegative() && right.isNegative())
        {
            std::vector<int> common_variables, left_variables,
right_variables;

            std::vector<IPulse*> left_buffer = left.getvalues();
            std::vector<IPulse*> right_buffer = right.getvalues();
            std::vector<IUnit> left_exclusive = left.getVariables();
            std::vector<IUnit> right_exclusive =
right.getVariables();

            for (IPulse* lp : left_buffer)
            {
                bool common = false;
                for (IPulse* rp : right_buffer)
                {

```

```

        if (*lp == *rp)
        {
            common = true;
        }
        common_variables.push_back(dynamic_cast<Graph::UnitPulse*>(lp)->variable);
    }
    if (!common)
    {
        left_variables.push_back(dynamic_cast<Graph::UnitPulse*>(lp)->variable);
    }
    for (IPulse* rp : right_buffer)
    {
        int rv = dynamic_cast<Graph::UnitPulse*>(rp)-
>variable;
        bool common = false;
        for (int lv : common_variables)
        {
            if (lv == rv)
            {
                common = true;
            }
        }
        if (!common)
        {
            right_variables.push_back(rv);
        }
    }
    for (int lv : left_variables)
    {
        Graph::UnitPulse* lp = new Graph::UnitPulse(ANY,
lv);
        right_exclusive.push_back(*lp);
        definitiveSet.push_back(lp);
    }
    for (int rv : right_variables)
    {
        Graph::UnitPulse* lp = new Graph::UnitPulse(ANY,
rv);
        left_exclusive.push_back(*lp);
        definitiveSet.push_back(lp);
    }
    for (int v : common_variables)
    {
        definitiveSet.push_back(new
Graph::UnitPulse(ANY, v));
    }

    exclusionSet.push_back(left_exclusive);
    exclusionSet.push_back(right_exclusive);
    size = common_variables.size() + left_variables.size() +
right_variables.size();
}

bool AggregatedPulse::operator==(IPulse& pulse)
{
    std::vector<IUnit> target_variables = pulse.getVariables();
    std::vector<IUnit> variables = getVariables();
    for (IUnit& unit : variables)

```

```

{
    for (IUnit& target_unit : target_variables)
    {
        if (unit.variable == target_unit.variable)
        {
            return true;
        }
    }
    return false;
}

IPulse* AggregatedPulse::operator+ (IPulse& pulse)
{
    if (pulse == Graph::UnitPulse(CONFLICT, 0))
    {
        return &pulse;
    }
    if (pulse.type == PulseType::UnitPulse || (pulse.type ==
PulseType::Pulse) && pulse.isNegative())
    {
        if (pulse != *this)
        {
            std::vector<IPulse*> buffer = pulse.getvalues();
            buffer.push_back(this);
            return new Graph::Pulse(false, buffer);
        }

        IPulse* prev = NULL;
        IPulse* result = NULL;
        for (std::vector<IUnit> exclude : exclusionSet)
        {
            std::vector<IPulse*> buffer;
            for (IUnit variable : exclude)
            {
                buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
            }
            Graph::Pulse exclusion_pulse =
Graph::Pulse(true, buffer);
            if (prev == NULL)
            {
                result = pulse + exclusion_pulse;
            }
            else
            {
                result = *prev + exclusion_pulse;
                //delete prev;
            }

            if (*result == Graph::UnitPulse(CONFLICT, 0))
            {
                return result;
            }
            else
            {
                prev = result;
            }
        }
        return result;
    }
    else
    {
        if (pulse != *this)

```

```

{
    if (pulse.type == PulseType::AggregatedPulse)
    {
        std::vector<IPulse*> buffer;
        buffer.push_back(&pulse);
        buffer.push_back(this);
        return new Graph::Pulse(false, buffer);
    }
    else
    {
        std::vector<IPulse*> buffer =
pulse.getvalues();

        buffer.push_back(this);
        return new Graph::Pulse(false, buffer);
    }
}

AggregatedPulse* result = new AggregatedPulse();
std::vector<std::vector<IUnit>> left_buffer,
right_buffer;

std::vector<IUnit> left_padding, right_padding;
if (pulse.type == PulseType::Pulse)
{
    right_buffer.push_back(pulse.getVariables());
    for (IPulse* p : pulse.getvalues())
    {
        if (*p != *this)
        {
            right_padding.push_back(IUnit(ANY, dynamic_cast<Graph::UnitPulse*>(p)-
>variable));
        }
    }
}
else
{
    for (IPulse* exc : pulse.getvalues())
    {
        right_buffer.push_back(exc-
>getVariables());
        for (IPulse* p :
dynamic_cast<Graph::AggregatedPulse*>(&pulse)->definitiveSet)
        {
            if (*p != *this)
            {
                right_padding.push_back(IUnit(ANY, dynamic_cast<Graph::UnitPulse*>(p)-
>variable));
            }
        }
    }
}
for (std::vector<IUnit> vec : exclusionSet)
{
    left_buffer.push_back(vec);
}
for (IPulse* p : definitiveSet)
{
    if (*p != *this)
    {
        left_padding.push_back(IUnit(ANY,
dynamic_cast<Graph::UnitPulse*>(p)->variable));
    }
}
}

```

```

        for (std::vector<IUnit> vec : left_buffer)
        {
            for (auto p = right_padding.begin(); p !=
right_padding.end(); p++)
            {
                vec.push_back(*p);
            }
            result->exclusionSet.push_back(vec);
        }
        for (std::vector<IUnit> vec : right_buffer)
        {
            for (auto p = left_padding.begin(); p !=
left_padding.end(); p++)
            {
                vec.push_back(*p);
            }
            result->exclusionSet.push_back(vec);
        }
        for (IPulse* p : definitiveSet)
        {
            result->definitiveSet.push_back(p);
            result->size++;
        }
        for (IUnit u : right_padding)
        {
            result->definitiveSet.push_back(new
Graph::UnitPulse(u.value, u.variable));
            result->size++;
        }

        long int total_weight = 0;
        for (std::vector<IUnit> vec : result->exclusionSet)
        {
            long int weight = 1;
            for (IUnit u : vec)
            {
                if (u.value == ANY)
                {
                    weight *= 2;
                }
            }
            total_weight += weight;
        }
        if (total_weight == pow(2, result->size))
        {
            delete result;
            return new Graph::UnitPulse(CONFLICT, 0);
        }
        else
        {
            return result;
        }
    }

}

std::vector<IPulse*> AggregatedPulse::getvalues()
{
    std::vector<IPulse*> result;
    for (std::vector<IUnit> exclude : exclusionSet)
    {
        std::vector<IPulse*> buffer;
        for (IUnit variable : exclude)
        {

```

```

        buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
    }
    result.push_back(new Graph::Pulse(true, buffer));
}
return result;
}

std::vector<IUnit> AggregatedPulse::getVariables()
{
    std::vector<IUnit> vec;

    for (IPulse* pulse : definitiveSet)
    {
        for (IUnit unit : pulse->getVariables())
        {
            vec.push_back(unit);
        }
    }

    return vec;
}

IPulse* AggregatedPulse::open()
{
    return new Graph::Pulse(false, definitiveSet);
}

std::string AggregatedPulse::print()
{
    std::string result;
    result += "Not { ";
    for (int i = 0; i < exclusionSet.size(); i++)
    {
        result += "( ";
        std::vector<IPulse*> buffer;
        for (IUnit variable : exclusionSet[i])
        {
            buffer.push_back(new
Graph::UnitPulse(variable.value, variable.variable));
        }
        result += Graph::Pulse(false, buffer).print();
        result += " )";
        if (i < exclusionSet.size() - 1)
        {
            result += ", ";
        }
    }
    result += " }";
    return result;
}

bool AggregatedPulse::isNegative()
{
    return true;
}

}

```

Utilities.h

Original:

```
#pragma once
#include "dependancies.h"

namespace Utilities
{
    class ThreadPool
    {
        static std::mutex mutex;
        static std::condition_variable cv;
        static std::queue<std::function<void()>> tasks;

        static int tasks_reported;
        static int tasks_completed;

        static bool solved;

        static struct Action
        {
            Action();

            void operator()();
        };

        static void report_task();
        static void report_task_completion();

    public:

        static int thread_count;

        static void initialise(int = 0);

        static void make_thread(std::function<void()>);

        static bool wait_until_done();

        static void problem_solved();

    };

    class Stack
    {
    public:

        static struct Entry
        {
            const int value;
            Entry* prev;

            Entry(int, Entry*);

            Entry(int);

            Entry(Entry*);
        };

        Stack();

        Stack(Stack*);
    };
}
```

```

        Stack& operator=(Stack&);

        int top();

        Entry* top_entry();

        void push(int);

        Entry* pop();

        bool empty();

    private:
        Entry* entry;
};

struct Unsolvable : public std::exception
{
    Unsolvable();
    const char* what();
};

struct AlwaysTrue : public std::exception
{
    AlwaysTrue();
    const char* what();
};

struct AlwaysFalse : public std::exception
{
    AlwaysFalse();
    const char* what();
};
}

```

Final:

```

#pragma once
#include "dependancies.h"

namespace Utilities
{
    class ThreadPool
    {
        static std::mutex mutex;
        static std::condition_variable cv;
        static std::queue<std::function<void()>> tasks;

        static int tasks_reported;
        static int tasks_completed;

        static bool solved;

        static struct Action
        {
            Action();

            void operator()();
        };
};

```



```

        static void report_task();

        static void report_task_completion();

public:
        static int thread_count;

        static void initialise(int = 0);

        static void make_thread(std::function<void()>);

        static void wait_until_done();

        static void problem_solved();

};

class Stack
{
public:
        static struct Entry
        {
                const int value;
                Entry* prev;

                Entry(int, Entry*);

                Entry(int);

                Entry(Entry*);
        };

        Stack();

        Stack(Stack*);

        Stack& operator=(Stack&);

        int top();

        Entry* top_entry();

        void push(int);

        Entry* pop();

        bool empty();

private:
        Entry* entry;
};

struct Unsolvable : public std::exception
{
        Unsolvable();
        const char* what();
};

struct AlwaysTrue : public std::exception
{
        AlwaysTrue();
        const char* what();
};

```

```

};

struct AlwaysFalse : public std::exception
{
    AlwaysFalse();
    const char* what();
};
}

```

ThreadPool.cpp

Original:

```

#include "utilities.h"

namespace Utilities
{
    ThreadPool::Action::Action()
    {
        report_task();
    }

    void ThreadPool::Action::operator()()
    {
        std::unique_lock<std::mutex> lock(mutex);
        std::function<void()> action;
        while (!solved)
        {
            if (tasks.empty())
            {
                cv.wait(lock, []() {!tasks.empty(); });
            }
            else
            {
                action = tasks.front();
                tasks.pop();
                lock.unlock();
                action();
                report_task_completion();
                if (thread_count == 0)
                {
                    break;
                }
            }
        }
    }

    void ThreadPool::report_task()
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        tasks_reported++;
    }

    void ThreadPool::report_task_completion()
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        tasks_completed++;
        cv.notify_all();
    }

    void ThreadPool::initialise(int number_of_threads)

```

```

{
    thread_count = number_of_threads;
    tasks = std::queue<std::function<void()>>();
    tasks_completed = 0;
    tasks_reported = 0;
    solved = false;

    for (int i = 1; i <= thread_count; i++)
    {
        std::thread(new ThreadPool::Action);
    }
}

void ThreadPool::make_thread(std::function<void()> action)
{
    std::lock_guard<std::mutex> synchronise(mutex);

    tasks.push(action);
    if (thread_count == 0)
    {
        std::thread(new ThreadPool::Action);
    }
}

bool ThreadPool::wait_until_done()
{
    std::unique_lock<std::mutex> lock(mutex);
    cv.wait(lock, []() {return tasks_completed > 0 &&
tasks_completed == tasks_reported; });

    tasks_completed = 0;
    tasks_reported = 0;

    lock.release();
}

void ThreadPool::problem_solved()
{
    {
        std::unique_lock<std::mutex> lock(mutex);

        solved = true;
    }
    cv.notify_all();
}
}

```

Final:

```

#include "utilities.h"

namespace Utilities
{
    ThreadPool::Action::Action()
    {}

    void ThreadPool::Action::operator()()
    {
        std::function<void()> action;
        while (true)

```

```

        {
            std::unique_lock<std::mutex> lock(mutex);
            cv.wait(lock, []() {return solved || !tasks.empty(); });
            if (solved)
            {
                break;
            }
            else if (!tasks.empty())
            {
                action = tasks.front();
                tasks.pop();
            }
            lock.unlock();

            action();

            report_task_completion();
        }
    }

    int ThreadPool::thread_count = 0;
    std::queue<std::function<void()>> ThreadPool::tasks =
std::queue<std::function<void()>>();
    int ThreadPool::tasks_completed = 0;
    int ThreadPool::tasks_reported = 0;
    bool ThreadPool::solved = false;
    std::mutex ThreadPool::mutex;
    std::condition_variable ThreadPool::cv;

    void ThreadPool::report_task()
    {
        tasks_reported++;
    }

    void ThreadPool::report_task_completion()
    {
        {
            std::lock_guard<std::mutex> synchronise(mutex);

            tasks_completed++;
        }
        std::cout << "tasks completed: " << tasks_completed <<
std::endl;
        std::cout << "tasks reported: " << tasks_reported << std::endl;
        std::cout << "tasks in queue: " << tasks.size() << std::endl;
        cv.notify_all();
    }

    void ThreadPool::initialise(int number_of_threads)
    {
        thread_count = number_of_threads;
        std::cout << "number of thread: " << thread_count << std::endl;;

        for (int i = 1; i <= thread_count; i++)
        {
            std::thread thread(std::bind([](Action* p) {p-
>operator()(); }, new Action()));
            thread.detach();
        }
    }

    void ThreadPool::make_thread(std::function<void()> action)

```

```

    {
        {
            std::lock_guard<std::mutex> synchronise(mutex);

            tasks.push(action);
            report_task();
            if (thread_count == 0)
            {
                std::thread thread(std::bind([](Action* p) {p-
>operator>()(); }, new Action()));
                thread.detach();
            }
        }
        cv.notify_all();
    }

    void ThreadPool::wait_until_done()
    {
        std::unique_lock<std::mutex> lock(mutex);
        cv.wait(lock, []() {return tasks_completed ==
tasks_reported; });

        tasks_completed = 0;
        tasks_reported = 0;
        std::cout << "done" << std::endl;
        lock.unlock();
    }

    void ThreadPool::problem_solved()
    {
        {
            std::unique_lock<std::mutex> lock(mutex);

            solved = true;
        }
        cv.notify_all();
    }
}

```

NAnd.cpp

Original:

```
#include "Alexandra.h"
```

```
namespace Graph
```

```
{
    NAnd::NAnd(int me, Graph::Node* left, Graph::Node* right) : Node(me),
    left(left), right(right) {}

```

```
    void NAnd::propagate(Utilities::Stack sat_trace, int last_node)
    {

```

```
Utilities::ThreadPool::make_thread(Graph::Propagate(Utilities::Stack(sat_trace),
me, right));

```

```
        left->propagate(sat_trace, me);
    }
}

```

Final:

```
#include "Alexandra.h"

namespace Graph
{
    NAnd::NAnd(int me, Graph::Node* left, Graph::Node* right) : Node(me),
    left(left), right(right) {}

    void NAnd::propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace)
    {
        std::cout << me << " " << left<< " " << right << std::endl;

        Utilities::ThreadPool::make_thread(Graph::Propagate(Utilities::Stack(sat_trace),
        Utilities::Stack(latch_trace), right));
        left->propagate(sat_trace, latch_trace);
    }
}
```

NNot.cpp

Original:

```
#include "Alexandra.h"

namespace Graph
{
    NNot::NNot(int me, Node* fanin) : Node(me), fanin(fanin) {}

    void NNot::propagate(Utilities::Stack sat_trace, int last_node)
    {
        sat_trace.push(me);
        fanin->propagate(sat_trace, me);
    }
}
```

Final:

```
#include "Alexandra.h"

namespace Graph
{
    NNot::NNot(int me, Node* fanin) : Node(me), fanin(fanin) {}

    void NNot::propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace)
    {
        std::cout << me << " NOT " << fanin << std::endl;
        sat_trace.push(me);
        fanin->propagate(sat_trace, latch_trace);
    }
}
```

NLatch.cpp

Original:

```
#include "Alexandra.h"

namespace Graph
{
    NLatch::NLatch(int me, int next_state, Node* next_state_function,
        Accumulator* accumulator) : Node(me), next_state(next_state),
        fanin(next_state_function), accumulator(accumulator) {}

    void NLatch::propagate(Utilities::Stack sat_trace, int last_node)
    {
        if (last_node < next_state)
        {
            if ((last_node == me + 1))
            {
                sat_trace.pop();
                accumulator->accumulate(sat_trace, me, true);
            }
            else
            {
                accumulator->accumulate(sat_trace, me, false);
            }
        }
        else
        {
            fanin->propagate(sat_trace, me);
        }
    }
}
```

Final:

```
#include "Alexandra.h"

namespace Graph
{
    NLatch::NLatch(int me, int next_state, Node* next_state_function,
        Accumulator* accumulator) : Node(me), next_state(next_state),
        fanin(next_state_function), accumulator(accumulator) {}

    void NLatch::propagate(Utilities::Stack sat_trace, Utilities::Stack
        latch_trace)
    {
        std::cout << me << " " << next_state << " " << fanin <<
        std::endl;
        bool visited = false;
        Utilities::Stack::Entry* iterator = latch_trace.top_entry();

        while (iterator != NULL)
        {
            if (iterator->value == me)
            {
                visited = true;
                break;
            }
            else
        }
```

```

        {
            iterator = iterator->prev;
        }
    }

    if (visited)
    {
        if ((sat_trace.top() == me + 1))
        {
            sat_trace.pop();
            accumulator->accumulate(sat_trace, 0, true);
        }
        else
        {
            accumulator->accumulate(sat_trace, 0, false);
        }
    }
    else
    {
        latch_trace.push(me);
        fanin->propagate(sat_trace, latch_trace);
    }
}
}
}

```

NVariable.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    NVariable::NVariable(int me, Accumulator* accumulator):Node(me),
    accumulator(accumulator){}

    void NVariable::propagate(Utilities::Stack sat_trace, int last_node)
    {
        if ((last_node == me + 1))
        {
            sat_trace.pop();
            accumulator->accumulate(sat_trace, me, true);
        }
        else
        {
            accumulator->accumulate(sat_trace, me, false);
        }
    }
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{

```



```

    NVariable::NVariable(int me, Accumulator* accumulator):Node(me),
    accumulator(accumulator){}

    void NVariable::propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace)
    {
        if ((sat_trace.top() == me + 1))
        {
            sat_trace.pop();
            accumulator->accumulate(sat_trace, me, true);
        }
        else
        {
            accumulator->accumulate(sat_trace, me, false);
        }
    }
}

```

Node.cpp

Original:

```

#include "graph.h"

namespace Graph
{
    void Node::propogate(Utilities::Stack, int) {}

    Node::Node(int me) : me(me){}
}

```

Final:

```

#include "graph.h"

namespace Graph
{
    void Node::propagate(Utilities::Stack, Utilities::Stack) {}

    Node::Node(int me) : me(me){}
}

```

Propagate.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    Propagate::Propagate(Utilities::Stack sat_trace, int last_node, Node*
    target) : sat_trace(sat_trace), last_node(last_node), target(target){}

    void Propagate::operator>()()
    {

```

```

        target->propagate(sat_trace, last_node);
    }
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{
    Propagate::Propagate(Utilities::Stack sat_trace, Utilities::Stack
    latch_trace, Node* target) : sat_trace(sat_trace), latch_trace(latch_trace),
    target(target){}

    void Propagate::operator()()
    {
        std::cout << "starting propagation" << std::endl;
        target->propagate(sat_trace, latch_trace);
    }
}

```

StartPoint.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    StartPoint::StartPoint(Node** nodes, int size) :IStartPoint(),
    fanins(fanins), size(size) {}
    StartPoint::StartPoint(Node** nodes) :IStartPoint(), fanins(fanins),
    size(1) {}

    void StartPoint::start()
    {
        Utilities::Stack sat_trace(0);
        if (size == 1)
        {
            Utilities::ThreadPool::make_thread(Propagate(sat_trace,
0, fanins[0]));
        }
        else
        {
            for (int i = 0; i < size; i++)
            {
                Utilities::Stack sub_trace(sat_trace);
                sub_trace.push(-(i + 1));

                Utilities::ThreadPool::make_thread(Propagate(sub_trace, 0, fanins[i]));
            }
            Utilities::ThreadPool::wait_until_done();
        }
    }
}

```

Final:

```
#include "Alexandra.h"

namespace Graph
{
    StartPoint::StartPoint(Node** nodes, int size) :IStartPoint(),
    fanins(nodes), size(size) {}
    StartPoint::StartPoint(Node** nodes) :IStartPoint(), fanins(nodes),
    size(1) {}

    void StartPoint::start()
    {
        Utilities::Stack sat_trace;
        Utilities::Stack latch_trace;
        if (size == 1)
        {
            Utilities::ThreadPool::make_thread(Propagate(sat_trace,
latch_trace, fanins[0]));
        }
        else
        {
            for (int i = 0; i < size; i++)
            {
                Utilities::Stack sub_trace(sat_trace);
                sub_trace.push(-(i + 1));

                Utilities::ThreadPool::make_thread(Propagate(sub_trace, latch_trace,
fanins[i]));
            }
            Utilities::ThreadPool::wait_until_done();
        }
    }
}
```

NodeHandler.cpp

Original:

```
#include "Alexandra.h"

namespace Graph
{
    NodeHandler::NodeHandler() :Node(0), node(NULL) {}

    NodeHandler::NodeHandler(Node* node) : Node(0), node(node) {}

    void NodeHandler::propagate(Utilities::Stack sat_trace, int last_node)
    {
        if (node != NULL)
        {
            node->propogate(sat_trace, last_node);
        }
        else
        {
            std::cout << "Empty Handler" << std::endl;
        }
    }

    void NodeHandler::set_node(Node* node)
    {

```

```

        this->node = node;
    }
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{
    NodeHandler::NodeHandler() : Node(0), node(NULL) {}

    NodeHandler::NodeHandler(Node* node) : Node(0), node(node) {}

    void NodeHandler::propagate(Utilities::Stack sat_trace, Utilities::Stack
latch_trace)
    {
        if (node != NULL)
        {
            node->propagate(sat_trace, latch_trace);
        }
        else
        {
            std::cout << "Empty Handler" << std::endl;
        }
    }

    void NodeHandler::set_node(Node* node)
    {
        this->node = node;
    }
}

```

Accumulator.cpp

Original:

```

#include "Alexandra.h"

namespace Graph
{
    Accumulator::AccNode::AccNode(bool is_master) : master(is_master),
signals(), nodes(), mutex() {}

    void Accumulator::AccNode::add_pulse(UnitPulse* signal)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        if (signal->value == CONFLICT)
        {
            signals.erase(signals.begin(), signals.end());
            signals.push_back(signal);
        }
        else if (signal->variable == 0 && signal->value == TRUE)
        {
            if (master)
            {
                signals.erase(signals.begin(), signals.end());
                signals.push_back(new UnitPulse(CONFLICT, 0));
            }
        }
    }
}

```

```

        }
    }
    else
    {
        for (IPulse* s : signals)
        {
            if ((s->toUnitPulse()->variable != signal-
>variable && s->toUnitPulse()->value != signal->value) && s->toUnitPulse()-
>value != CONFLICT)
            {
                signals.push_back(signal);
            }
            else
            {
                signals.erase(signals.begin(),
signals.end());
                signals.push_back(new
UnitPulse(CONFLICT, 0));
            }
        }
    }

    void Accumulator::AccNode::add_node(AccNode* node)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        nodes.push_back(node);
    }

    IPulse* Accumulator::AccNode::fold()
    {
        if (nodes.empty())
        {
            return new Pulse(master, signals);
        }
        else
        {
            IPulse* result;
            Add* new_adder = new Add(signals.size() + 1, NULL,
result);
            for (AccNode* n : nodes)
            {
                Utilities::ThreadPool::make_thread(Fold(new_adder, n));
            }
            new_adder->add_operand(new Pulse(master, signals));
            Utilities::ThreadPool::make_thread(*new_adder);
            Utilities::ThreadPool::wait_until_done();
            return result;
        }
    }

    Accumulator::Accumulator() : mutex(), accNodes(), master_pointer(NULL)
{}

    Accumulator::AccNode* Accumulator::get_node(Utilities::Stack::Entry* id)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        if (accNodes.contains(id))
        {
            return accNodes[id];
        }
    }

```

```

        return NULL;
    }

    void Accumulator::add_node(Utilities::Stack::Entry* id,
Accumulator::AccNode* node)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        accNodes[id] = node;
    }

    void Accumulator::accumulate(Utilities::Stack sat_trace, int variable,
bool state)
    {
        AccNode* prev_node = NULL;
        while (!sat_trace.empty())
        {
            AccNode* node;
            Utilities::Stack::Entry* current_not = sat_trace.pop();
            if (prev_node == NULL)
            {
                node = get_node(current_not);
                if (node == NULL)
                {
                    node = new AccNode(current_not->value ==
0);

                    add_node(current_not, node);

                    node->add_pulse(new UnitPulse((state)?(TRUE):
(FALSE), variable));
                }
            }
            else
            {
                node = get_node(current_not);
                if (node == NULL)
                {
                    node = new AccNode(current_not->value ==
0);

                    add_node(current_not, node);
                    node->add_node(prev_node);
                }
            }
            prev_node = node;
            if (master_pointer == NULL && current_not->value == 0)
            {
                master_pointer = current_not;
            }
        }

        IPulse* Accumulator::solve()
        {
            return get_node(master_pointer)->fold();
        }

        IPulse* Accumulator::solve(std::map<int, bool>& assumption)
        {
            std::vector<IPulse*> buffer;
            for (auto iter = assumption.begin(); iter != assumption.end();
iter++)
            {

```

```

        buffer.push_back(new UnitPulse((iter->second) ? (TRUE) :
(FALSE), iter->first));
    }
    return (*solve() + *(new Pulse(false, buffer)));
}

Accumulator::Add::Add(int size, Add* prev, IPulse* result) : mutex(),
cv(), number_of_operands(size), adder(prev), result_buffer(&result), operands()
{}

void Accumulator::Add::add_operand(IPulse* pulse)
{
    std::lock_guard<std::mutex> synchronise(mutex);

    operands.push_back(pulse);
}

void Accumulator::Add::operator()()
{
    std::unique_lock<std::mutex> lock(mutex);

    if (operands.size() < number_of_operands)
    {
        if (Utilities::ThreadPool::thread_count > 0)
        {
            lock.unlock();
            Utilities::ThreadPool::make_thread(*this);
            return;
        }
        else
        {
            struct Wait
            {
                const int size;
                std::vector<IPulse*>& vec;
                Wait(int size, std::vector<IPulse*>&
vec) : size(size), vec(vec){}

                bool operator()()
                {
                    return size == vec.size();
                }
            };

            cv.wait(lock, Wait(number_of_operands,
operands));
        }
    }

    IPulse* garbage = NULL;
    IPulse* result = NULL;
    for (IPulse* p : operands)
    {
        if (result == NULL)
        {
            result = p;
        }
        else
        {
            IPulse* intermideate = *p + *result;
            delete garbage;
            result = intermideate;
            garbage = intermideate;
        }
    }
    lock.unlock();
}

```

```

        *result_buffer = result;
    }

    Accumulator::Fold::Fold(Add* adder, AccNode* node) : adder(adder),
node(node){}

    void Accumulator::Fold::operator()()
    {
        if (node->nodes.empty())
        {
            adder->add_operand(new Pulse(node->master, node-
>signals));
        }
        else
        {
            Add* new_adder = new Add(node->signals.size() + 1,
adder);
            for (AccNode* n : node->nodes)
            {
                Utilities::ThreadPool::make_thread(Fold(new_adder, n));
            }
            new_adder->add_operand(new Pulse(node->master, node-
>signals));
            Utilities::ThreadPool::make_thread(*new_adder);
        }
    }
}

```

Final:

```

#include "Alexandra.h"

namespace Graph
{
    Accumulator::AccNode::AccNode() : master(false), signals(),
next_node(NULL), child_nodes(0), fold_count(0), intermediate_value() {}

    void Accumulator::AccNode::add_child()
    {
        child_nodes++;
    }
    void Accumulator::AccNode::set_master()
    {
        master = true;
    }

    void Accumulator::AccNode::ripe(IPulse& pulse)
    {
        bool is_ripe;
        {
            std::lock_guard<std::mutex> synchronise(mutex);

            if (intermediate_value == NULL)
            {
                intermediate_value = new Pulse(!master,
signals);
            }

            IPulse* new_value = *intermediate_value + pulse;
            //delete intermediate_value;

```



```

        intermediate_value = new_value;
        fold_count++;
        is_ripe = fold_count == child_nodes;
    }

    std::cout << "intermediate value at parent: " <<
intermediate_value->print() << std::endl;

    if (is_ripe)
    {
        Utilities::ThreadPool::make_thread(Fold(this));
    }
}

void Accumulator::AccNode::add_pulse(UnitPulse* signal)
{
    std::lock_guard<std::mutex> synchronise(mutex);

    bool ignore = false;
    if (signal->variable != 0 && signal->variable != 1)
    {
        for (IPulse* s : signals)
        {
            UnitPulse* up = dynamic_cast<UnitPulse*>(s);
            if ((up->value == CONFLICT) || (up->variable ==
signal->variable && up->value == signal->value))
            {
                ignore = true;
            }
            else if (up->variable == signal->variable && up-
>value != signal->value)
            {
                signals.erase(signals.begin());
                signals.push_back(new
UnitPulse(CONFLICT, 0));

                ignore = true;
            }
        }
    }
    else
    {
        if (!master)
        {
            ignore = true;
        }
        else
        {
            if ((signal->variable == 0 && signal->value ==
TRUE))
            {
                signals.erase(signals.begin());
                signals.push_back(new
UnitPulse(CONFLICT, 0));

                ignore = true;
            }
        }
    }

    if (ignore)
    {
        delete signal;
    }
    else
    {

```

```

        signals.push_back(signal);
    }
}

void Accumulator::AccNode::add_node(AccNode* node)
{
    next_node = node;
}

void Accumulator::AccNode::fold()
{
    {
        std::cout << "starting folding at master: " << master <<
std::endl;
        std::lock_guard<std::mutex> synchronise(mutex);

        if (intermediate_value == NULL)
        {
            if (signals.size() == 1)
            {
                intermediate_value = signals.front();
            }
            else
            {
                intermediate_value = new Pulse(!master,
signals);
            }
        }

        std::cout << "intermediate value: " <<
intermediate_value->print() << std::endl;

        if (!master)
        {
            next_node->ripe(*intermediate_value);
        }
    }

    IPulse* Accumulator::AccNode::get_result()
    {
        return intermediate_value;
    }

    Accumulator::Accumulator() : mutex(), accNodes(), master_pointer(NULL)
{}

    Accumulator::AccNode* Accumulator::get_node(Utilities::Stack::Entry* id)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        if (accNodes.contains(id))
        {
            return accNodes[id];
        }
        return NULL;
    }

    void Accumulator::add_node(Utilities::Stack::Entry* id,
Accumulator::AccNode* node)
    {
        std::lock_guard<std::mutex> synchronise(mutex);

        if (id->value == 0)

```

```

        {
            node->set_master();
            master_pointer = id;
        }

        accNodes[id] = node;
    }

    void Accumulator::accumulate(Utilities::Stack sat_trace, int variable,
bool state)
    {
        UnitPulse* signal = new UnitPulse((state) ? (TRUE) : (FALSE),
variable);

        AccNode* current_node;
        AccNode* prev_node = NULL;
        AccNode* leaf_node = NULL;

        Utilities::Stack::Entry* iterator = sat_trace.top_entry();

        bool complete = false;

        while (!complete && iterator != NULL)
        {
            current_node = get_node(iterator);
            if (current_node == NULL)
            {
                current_node = new AccNode();
                add_node(iterator, current_node);
            }

            if (leaf_node == NULL)
            {
                leaf_node = current_node;
                leaf_node->add_pulse(signal);
            }

            if (prev_node != NULL)
            {
                current_node->add_child();
                prev_node->add_node(current_node);
            }

            if (current_node->next_node == NULL)
            {
                prev_node = current_node;
            }
            else
            {
                complete = true;
            }

            iterator = iterator->prev;
        }
    }

    IPulse* Accumulator::solve()
    {
        for (auto p = accNodes.begin(); p != accNodes.end(); p++)
        {
            if (p->second->child_nodes == 0)
            {

```

```

        Utilities::ThreadPool::make_thread(Fold(p-
>second));
    }
    Utilities::ThreadPool::wait_until_done();
    return accNodes[master_pointer]->get_result();
}

IPulse* Accumulator::solve(std::map<int, bool>& assumption)
{
    std::vector<IPulse*> buffer;
    for (auto iter = assumption.begin(); iter != assumption.end();
iter++)
    {
        buffer.push_back(new UnitPulse((iter->second) ? (TRUE) :
(FALSE), iter->first));
    }
    return (*solve() + *(new Pulse(false, buffer)));
}

Accumulator::Fold::Fold(AccNode* node) : node(node) {}

void Accumulator::Fold::operator>()()
{
    node->fold();
}
}

```

References and Acknowledgements

[1] Boolean Satisfiability Problem on Wikipedia

https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

[2] Davis-Putnam-Logemann-Loveland algorithm on Wikipedia

https://en.wikipedia.org/wiki/DPLL_algorithm

[3] Conflict-Driven Clause Learning on Wikipedia

https://en.wikipedia.org/wiki/Conflict-driven_clause_learning

[4] And-Inverter Graph on Wikipedia

https://en.wikipedia.org/wiki/And-inverter_graph

[5] AIGER format on GitHub – Armin Biere et. al.

<https://github.com/arminbiere/aiger/blob/master/FORMAT>

[6] “Extending SAT Solvers to Cryptographic Problems” Mate Soos et. al. (2009)

- [7] “An Extensible SAT Solver” Niklas Een and Niklas Sörenson, version 1.2 (2003)
- [8] “A Circuit-Based SAT Solver For Logic Synthesis” He-Teng Zhang et. al. (2021)
- [9] “SAT-Sweeping Enhanced For Logic Synthesis” Luca Amaru et. al. (2020)
- [10] “Deep Integration of Circuit Simulator and SAT Solver” He-Teng Zhang et. al. (2021)
- [11] Conjunctive-Normal Form on Wikipedia

https://en.wikipedia.org/wiki/Conjunctive_normal_form